



Facultad de Ciencias

Búsqueda de factores RSA compartidos en los registros de Transparencia de Certificados: Un enfoque basado en el algoritmo Binary Tree Batch GCD

Finding shared RSA factors in the Certificate Transparency logs: A Binary Tree Batch GCD approach

Trabajo de fin de máster
para acceder al

**MÁSTER INTERUNIVERSITARIO (UC-UIMP) EN DATA
SCIENCE**

Autor: David Quinzaños Saiz
Codirector: Ana I. Gómez Pérez
Codirector: Domingo Gómez Pérez

Junio de 2026

Agradecimientos

Quiero aprovechar esta página para agradecer su compañía y apoyo a todas las personas que me han acompañado durante este curso, tanto a mi familia, pareja y amigos, como a mis tutores, compañeros y profesores. Ha sido un año muy distinto, pero he aprendido muchas cosas, he continuado creciendo y me alegro de poder cerrarlo de la mejor manera posible: en casa y con toda mi gente cerca.

Resumen

palabras clave: RSA, batch GCD, Transparencia de Certificados, criptografía, ciberseguridad

La seguridad en Internet depende del uso adecuado de los servicios de certificación, que permiten la autenticación de la identidad digital. *Certificate Transparency* es un estándar de Internet que obliga a las autoridades certificadoras a publicar los certificados emitidos en registros públicos, verificables y de solo anexo de manera que puedan ser auditados públicamente. La emisión masiva de certificados implica generar un número muy alto de módulos Rivest, Shamir y Adleman (RSA), por lo que puede aparecer una vulnerabilidad cuando la generación de claves no dispone de suficiente aleatoriedad y se reutilizan números primos. Por ello, algunos ataques se basan en la detección de factores compartidos en claves públicas, lo que motiva la auditoría periódica de los certificados publicados en CT en busca de claves débiles.

Para analizar conjuntos completos de claves se emplean técnicas denominadas *batch Greatest Common Divisor (GCD)*, que reorganizan los cálculos para buscar divisores comunes. En este trabajo analizamos una variante basada en un árbol binario, el *Binary Tree Batch GCD*, y con este propósito hemos desarrollado una implementación propia en C++17 con la biblioteca *GNU Multiple Precision Arithmetic Library* (GMP). Hemos comparado su rendimiento con el de una implementación Python de referencia, para posteriormente aplicar nuestra implementación a módulos RSA reales descargados del repositorio público *ct-archive*. Los resultados muestran que *Binary Tree Batch GCD* mejora de forma consistente al método clásico basado en *remainder tree* y que la implementación C++/GMP reduce de manera significativa tanto el tiempo de ejecución como el consumo de memoria.

Como paso final, hemos aplicado el procedimiento a certificados reales de emisores obtenidos de *ct-archive*. Para ello, se extrajeron los módulos RSA con OpenSSL, se normalizaron en hexadecimal y se deduplicaron antes del análisis. El conjunto final contiene 54.278 módulos únicos brutos, de los cuales 54.275 se consideran plausibles tras la validación. La ejecución no detectó factores compartidos no triviales ni claves vulnerables en este subconjunto, con un tiempo de ejecución del algoritmo sobre el conjunto de aproximadamente 23,8 segundos, frente a los tiempos en minutos de las implementaciones de referencia. El trabajo combina así desarrollo algorítmico, evaluación experimental y validación sobre datos reales.

Abstract

keywords: RSA, batch GCD, Certificate Transparency, cryptography, cybersecurity

Internet security depends on the proper use of certification services, which enable the authentication of digital identities. *Certificate Transparency* (CT) is an Internet standard that requires certification authorities to publish issued certificates in public, verifiable, append-only logs so that they can be publicly audited. The large-scale issuance of certificates involves the generation of a very high number of Rivest, Shamir and Adleman (RSA) moduli. As a result, a vulnerability may arise when key generation does not provide sufficient randomness and prime numbers are reused. For this reason, some attacks are based on detecting shared factors in public keys, which motivates the periodic auditing of certificates published in CT logs in order to identify weak keys.

To analyse complete sets of keys, techniques known as *batch Greatest Common Divisor (GCD)* are used, which reorganise the computations required to search for common divisors. In this work, we analyse a variant based on a binary tree, the *Binary Tree Batch GCD*. For this purpose, we have developed our own implementation in C++17 using the *GNU Multiple Precision Arithmetic Library* (GMP). We have compared its performance with that of a reference Python implementation, and subsequently applied our implementation to real RSA moduli downloaded from the public *ct-archive* repository. The results show that *Binary Tree Batch GCD* consistently improves upon the classical method based on a *remainder tree*, and that the C++/GMP implementation significantly reduces both execution time and memory consumption.

As a final step, we applied the procedure to real issuer certificates obtained from *ct-archive*. To this end, the RSA moduli were extracted using OpenSSL, normalised in hexadecimal format, and deduplicated before the analysis. The final dataset contains 54,278 raw unique moduli, of which 54,275 are considered plausible after validation. The execution did not detect any non-trivial shared factors or vulnerable keys in this subset. The algorithm processed the complete dataset in approximately 23.8 seconds, compared with execution times of several minutes for the reference implementations. Thus, this work combines algorithmic development, experimental evaluation, and validation on real-world data.

Índice

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos y alcance	4
1.3. Estructura del documento	5
2. Fundamentos teóricos	7
2.1. Fundamentos de RSA	7
2.2. Máximo común divisor y su relación con RSA	9
2.3. Entropía y generación de claves RSA	10
2.4. Certificate Transparency logs	12
3. Estado del arte	15
3.1. Detección de factores compartidos en claves públicas RSA	15
3.2. Análisis masivo de claves débiles en Internet	16
3.3. Enfoque clásico basado en product tree y remainder tree	17
3.4. Binary Tree Batch GCD como propuesta reciente	18
3.5. Encaje y motivación del presente trabajo	19
4. Metodología y algoritmo propuesto	21
4.1. Herramientas utilizadas	21
4.2. Fundamento del <i>batch GCD</i>	22
4.3. Algoritmo <i>Binary Tree Batch GCD</i>	23
4.4. Implementación desarrollada	26
4.5. Preparación de datos de CT-archive	27
5. Resultados y discusión	29
5.1. Entorno experimental	29
5.2. Configuración de la comparativa sintética	30
5.3. Validación funcional	31
5.4. Comparación de tiempos de ejecución	31
5.5. Comparación de consumo de memoria	32
5.6. Efecto del número de claves débiles	33
5.7. Resultados sobre datos reales	34
5.7.1. Extracción y consolidación de módulos reales	34
5.7.2. Ejecución del ataque sobre módulos reales	35
5.8. Discusión	36
6. Conclusiones y limitaciones	37
Referencias	39

Capítulo 1

Introducción

1.1. Contexto y motivación

La necesidad de proteger la información y mantener el secreto de las comunicaciones ha existido desde hace siglos. Ya en la Antigüedad se empleaban métodos sencillos para dificultar la lectura de los mensajes por parte de terceros, siendo los más conocidos el cifrado César y el criptosistema de Vigenère. Más adelante, durante la Segunda Guerra Mundial, se abrió la era computacional de la criptografía y máquinas como Enigma demostraron hasta qué punto esta disciplina podía ser decisiva en contextos militares y estratégicos. El trabajo de Alan Turing sentó muchas de las ideas fundamentales de la criptografía y la informática moderna, en las que la computación permite poner a prueba la seguridad de los sistemas criptográficos.

Sin embargo, todos estos métodos compartían una característica común: se trataba de sistemas de criptografía simétrica, es decir, emisor y receptor debían conocer previamente la misma clave secreta. Este enfoque puede funcionar en escenarios reducidos, pero plantea problemas evidentes cuando el número de usuarios crece. En el contexto actual, donde millones de personas se conectan continuamente a servicios en Internet, repartir de forma segura una misma clave entre todas las partes resulta inviable, considerando además que la seguridad de estos sistemas reside en el tamaño de clave en bits. Ejemplos de sistemas criptográficos modernos de clave simétrica son los criptosistemas de bloque como DES, 3DES y fundamentalmente AES [[National Institute of Standards and Technology, 2001b](#)].

Para dar respuesta a este problema surgió la criptografía asimétrica propuesta por Diffie-Hellman, basada en el uso de dos claves relacionadas matemáticamente: una clave pública, que puede compartirse libremente, y una clave privada, que debe mantenerse en secreto [[Rivest et al., 1978](#)]. Gracias a este modelo, hoy es posible establecer comunicaciones seguras sin necesidad de haber intercambiado previamente una clave secreta, tan solo mediante la publicación de la clave pública de forma confiable. Además, la criptografía asimétrica permite no solo cifrar información, sino también autenticar identidades y firmar digitalmente mensajes, por lo que los criptosistemas de clave pública se han convertido en una herramienta fundamental dentro de la seguridad informática.

La criptografía moderna combina estas primitivas asimétricas con algoritmos simétricos, funciones resumen, códigos de autenticación y protocolos diseñados para alcanzar propiedades concretas de confidencialidad, integridad y autenticación [[Menezes et al., 1996](#)]. En una conexión web actual, por ejemplo, la criptografía de clave pública permite autenticar al servidor y acordar secretos, mientras que el intercambio posterior de datos se protege con cifrado simétrico por su mayor eficiencia. Por ello, la seguridad práctica no depende de un único algoritmo, sino de la correcta composición del reparto de claves, uso de certificados electrónicos, protocolos e implementaciones.

Según [Aumasson \[2024\]](#), un certificado es una clave pública acompañada de una firma sobre ella, además de otra información asociada. Como veremos, estos certificados en Internet permiten un uso

de la Web seguro y confiable. Por ejemplo, para que un navegador web pueda confiar en una clave pública, es necesario que pueda verificar que esta pertenece a la entidad con la que se quiere comunicar (por ejemplo www.unican.es). En este caso es necesaria una firma electrónica que permita verificar la vinculación con el dominio, siendo este el papel de la entidad de certificación o «certificate authority» (CA). En el caso de la Universidad de Cantabria, la entidad certificadora puede consultarse desde el propio navegador web (ver Figura 1), ya que los navegadores incluyen de serie los certificados raíz de las autoridades reconocidas.

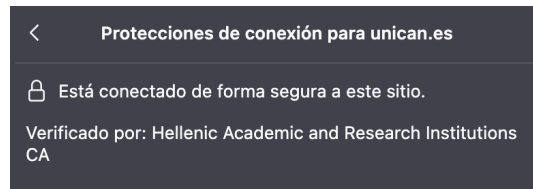


Figura 1: Ejemplo de entidad de certificación que acredita la autenticidad del dominio unican.es

El esquema de la evolución de los sistemas criptográficos de la Figura 2 no representa una sustitución completa de unas técnicas por otras, sino una ampliación progresiva de las herramientas disponibles. La aparición de la criptografía de clave pública permitió abordar problemas de distribución de claves y autenticación que limitaban a los sistemas anteriores, y preparó el terreno para el uso de RSA, TLS y los certificados electrónicos en la Web.

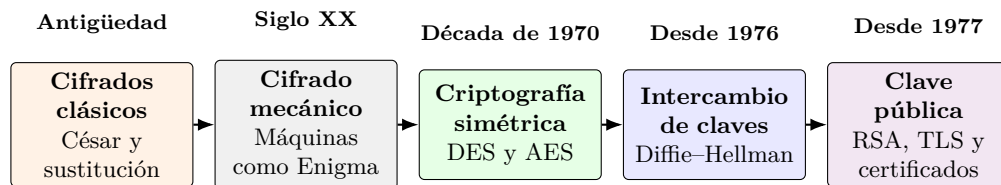


Figura 2: Evolución simplificada desde los cifrados clásicos hasta la clave pública.

Por otro lado, queremos introducir brevemente el protocolo de comunicación *Transport Layer Security* (TLS), que garantiza la confidencialidad de la comunicación extremo a extremo entre dos equipos conectados a través de Internet. En este protocolo se intercambian mensajes entre el cliente y el servidor para acordar una clave segura. En un primer mensaje el cliente envía un mensaje para establecer la conexión, con los métodos de cifrado disponibles y su clave pública generada para esa sesión. En un segundo mensaje, el servidor responde con un mensaje en el que se envía, entre otra información, el método de cifrado elegido, una clave pública propia y un certificado, junto a la información intercambiada firmada. Tras verificar toda la información recibida, el cliente podrá iniciar la comunicación cifrada. Un resumen del esquema de intercambio de mensajes se muestra en la Figura 3.

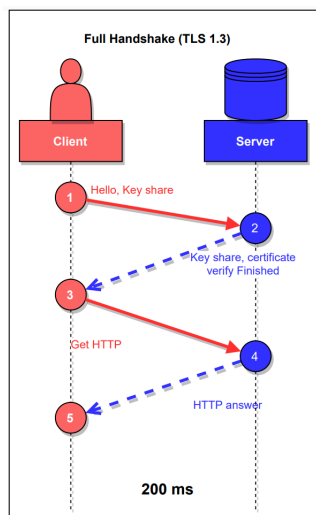


Figura 3: Esquema del intercambio (TLS 1.3). Fuente Halub3, CC BY-SA 4.0, via Wikimedia Commons

Por último, introduciremos el concepto de *Web Public Key Infrastructure* (Web PKI). Se trata del ecosistema que hace posible la confianza en los certificados de la Web: lo componen las autoridades certificadoras que emiten y firman certificados, los navegadores y sistemas operativos que mantienen almacenes de certificados raíz de confianza, los servidores que presentan sus certificados durante el establecimiento de conexiones TLS y los mecanismos de supervisión y revocación que vigilan el correcto funcionamiento del conjunto. En este ecosistema, la seguridad no depende de un único actor, sino de que cada componente cumpla su papel: una autoridad certificadora que emita certificados incorrectos, o una clave pública generada de forma deficiente, puede comprometer la confianza de toda la cadena. Precisamente para reforzar la supervisión de este ecosistema surge *Certificate Transparency*¹.

En este contexto, los *Certificate Transparency logs*² o *CT logs* constituyen una fuente de información especialmente interesante, ya que permiten acceder a grandes volúmenes de certificados reales emitidos para sitios web. Certificate Transparency es un ecosistema en el que los certificados están depositados en logs públicos y distribuidos, lo cual permite a diferentes operadores monitorizarlos de forma transparente. Esto los convierte en una base de datos muy valiosa para estudiar cómo se está desplegando realmente la criptografía en Internet y para detectar posibles debilidades en claves públicas utilizadas en la práctica.

Dentro de este ámbito, uno de los algoritmos más conocidos y utilizados históricamente ha sido el esquema RSA. Su seguridad se basa en la dificultad de factorizar números enteros grandes obtenidos como producto de dos números primos también grandes [Rivest et al., 1978]. En condiciones normales, esta dificultad hace que obtener la clave privada a partir de la clave pública resulte inviable en la práctica. Por ello, RSA ha ocupado durante muchos años un papel central dentro de la infraestructura de clave pública y de los certificados digitales utilizados en la Web. Aunque RSA está siendo desplazado progresivamente ante la amenaza de la computación cuántica, ya que el algoritmo de Shor permitiría factorizar módulos RSA de forma eficiente en un computador cuántico suficientemente grande [Shor, 1997], los organismos de estandarización prevén una transición gradual hacia la criptografía postcuántica [National Institute of Standards and Technology, 2024], por lo que RSA seguirá presente en sistemas de firma digital y certificados durante los próximos años.

No obstante, la seguridad de RSA no depende únicamente de su fundamento matemático, sino también de cómo se generan las claves en la práctica. Trabajos previos han mostrado que errores en este proceso pueden dar lugar a claves vulnerables, lo que convierte el análisis de implementaciones reales

¹<https://certificate.transparency.dev/howctworks/#pki>

²<https://certificate.transparency.dev/logs/>

en una cuestión especialmente relevante desde el punto de vista de la ciberseguridad [Heninger et al., 2012; Pelofske, 2024; Våge, 2022]. Los trabajos previos mostraron el interés de este enfoque y señalaron que todavía existe margen para ampliar el análisis sobre conjuntos de datos mayores [Våge, 2022]. Posteriormente se propuso el algoritmo *Binary Tree Batch GCD* como una alternativa más eficiente al enfoque clásico basado en *product tree* y *remainder tree* [Pelofske, 2024]. Disponer de algoritmos de detección cada vez más eficientes resulta, por tanto, esencial para auditar de forma continua la seguridad de los certificados desplegados en el ecosistema digital, y constituye la motivación central de este trabajo.

1.2. Objetivos y alcance

El objetivo general de este trabajo será estudiar la detección eficiente de factores primos compartidos entre módulos RSA y aplicar esta técnica a certificados obtenidos de fuentes públicas de *Certificate Transparency*. Para ello, se revisarán los fundamentos de esta vulnerabilidad y los métodos *batch GCD* utilizados para detectarla, prestando especial atención al algoritmo *Binary Tree Batch GCD* como alternativa al enfoque clásico basado en *product tree* y *remainder tree* que se detallarán en los capítulos posteriores.

A partir de esta base, se reproducirá, a una escala compatible con los recursos disponibles, la comparación experimental con una propuesta encontrada en la literatura. Se desarrollará una implementación propia en C++17, cuyas herramientas y decisiones técnicas se detallarán en el capítulo de metodología, y se comprobará que sus resultados coinciden con la implementación de referencia elegida. Se construirá además un flujo para extraer, normalizar, validar y deduplicar módulos RSA procedentes de certificados reales, y se aplicará la implementación al conjunto consolidado para estudiar las diferencias de tiempo de ejecución y consumo máximo de memoria. Los resultados se interpretarán de acuerdo con el alcance de los datos analizados y las limitaciones del diseño experimental.

El trabajo combinará así revisión teórica, reproducción experimental, desarrollo algorítmico y aplicación sobre datos reales. Desde el punto de vista de la ciberseguridad, permitirá estudiar una vulnerabilidad concreta pero con consecuencias graves, ya que una generación deficiente de claves puede comprometer completamente la seguridad de RSA. Al mismo tiempo, situará el análisis en un escenario aplicado, en el que no basta con que los algoritmos sean seguros en teoría, sino que también deben poder evaluarse de forma eficiente sobre claves utilizadas en la práctica.

La combinación de experimentos sintéticos y certificados reales responde a dos necesidades complementarias. Los primeros permiten comprobar el comportamiento del algoritmo en condiciones controladas y comparar las distintas implementaciones sobre entradas equivalentes; los segundos permiten valorar su utilidad sobre claves públicas procedentes de un entorno efectivamente desplegado. El alcance del trabajo une así validación experimental y aplicación práctica, sin pretender extrapolar los resultados más allá del conjunto de certificados finalmente analizado. Hemos incluido un esquema con el flujo seguido para el análisis de los CT logs en la Figura 4.

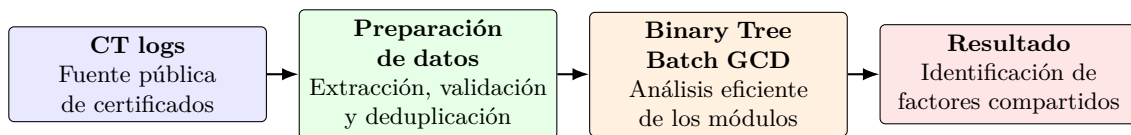


Figura 4: Visión global del problema y del flujo de trabajo seguido.

1.3. Estructura del documento

El resto del documento se ha organizado del siguiente modo. El Capítulo 2 presenta los fundamentos matemáticos necesarios, así como el funcionamiento del algoritmo RSA. Asimismo, se introducen las condiciones que pueden dar lugar a vulnerabilidades por reutilización de factores primos y, por último, se presentan los detalles técnicos de los certificados y del proyecto *Certificate Transparency*. El Capítulo 3 se centra en la revisión de los principales trabajos previos relacionados con la detección de claves RSA vulnerables. El Capítulo 4 presenta las herramientas utilizadas, el esquema de *Binary Tree Batch GCD*, la implementación propia desarrollada y la metodología seguida para la preparación de los datos. El Capítulo 5 expone y analiza los resultados obtenidos sobre conjuntos sintéticos y sobre el conjunto real consolidado. Finalmente, el Capítulo 6 recoge las conclusiones del trabajo y las limitaciones identificadas.

Capítulo 2

Fundamentos teóricos

En este capítulo presentaremos los conceptos teóricos necesarios para comprender el problema abordado en este Trabajo de Fin de Máster. Dado que nuestro objetivo es estudiar la detección de factores compartidos en claves RSA extraídas de *Certificate Transparency logs*, consideramos que los cuatro elementos fundamentales que es necesario introducir previamente son: el funcionamiento general de RSA, el papel del máximo común divisor en este contexto, la definición de la entropía respecto a la generación de claves y, por último, los fundamentos técnicos relativos a *CT logs* como fuente de datos para este tipo de análisis.

No se pretende realizar aquí una exposición exhaustiva de toda la teoría criptográfica relacionada con el funcionamiento de certificados en la Web, sino presentar únicamente aquellos conceptos que resultarán necesarios para entender el desarrollo posterior del trabajo. De este modo, en los capítulos siguientes nos centraremos en las propuestas realizadas, en la metodología empleada para el desarrollo del trabajo y en los detalles técnicos de la implementación del algoritmo utilizado.

2.1. Fundamentos de RSA

RSA es uno de los algoritmos más conocidos de criptografía asimétrica y uno de los más influyentes en la historia de la seguridad informática [Rivest et al., 1978].

Como se ha introducido en el capítulo anterior, a diferencia de la criptografía simétrica, donde emisor y receptor comparten la misma clave secreta, en criptografía asimétrica cada usuario dispone de un par de claves relacionadas matemáticamente: una clave pública p_k y una clave privada s_k .

Cualquier esquema de criptografía asimétrica tiene que definir tres algoritmos:

- Generación de claves: Una función $KeyGen(\lambda)$ que produce un par de claves (p_k, s_k) , dado un parámetro de seguridad λ .
- Encriptación: Una función $Enc(m, p_k)$, que, para un mensaje m , genera un mensaje cifrado c .
- Desencriptación: Una función $Dec(c, s_k)$ que, dado c , devuelve de nuevo el mensaje m .

Siempre que el esquema cumpla las condiciones de corrección, es posible enviar mensajes confidenciales a un usuario utilizando su clave pública p_k , de forma que solo la clave privada correspondiente s_k permita recuperar el mensaje original.

Este modelo permitió abordar uno de los grandes problemas de la criptografía simétrica tradicional: la necesidad de compartir previamente una clave secreta por un canal seguro. Gracias a la criptografía de clave pública, es posible establecer comunicaciones seguras, autenticar identidades y verificar firmas digitales en entornos abiertos como Internet, donde las partes no siempre han intercambiado

un secreto con anterioridad.

Esta misma separación entre clave pública y clave privada también permite construir esquemas de firma electrónica. En este caso, la idea no es ocultar el contenido del mensaje, sino demostrar su origen y garantizar que no ha sido modificado.

De forma simplificada, el firmante utiliza su clave privada para generar una firma asociada al mensaje, y cualquier verificador puede comprobar esa firma utilizando la clave pública correspondiente. Así, si la verificación es correcta, se obtiene una garantía criptográfica de que la firma ha sido generada con la clave privada asociada a esa clave pública, sin necesidad de revelar dicha clave. En la práctica, los esquemas de firma no se aplican normalmente sobre el mensaje completo, sino sobre un resumen criptográfico del mismo, pero esta descripción simplificada permite entender el papel que desempeña RSA en autenticación y firma digital. Para más detalles, nos remitimos a [Menezes et al. \[1996\]](#).

Para garantizar la seguridad de este tipo de sistemas, es especialmente importante que no se pueda deducir información útil de s_k conociendo únicamente p_k . En los criptosistemas de clave pública, esta propiedad se apoya normalmente en la dificultad computacional de resolver ciertos problemas matemáticos: aunque las operaciones necesarias para generar las claves, cifrar o verificar sean eficientes, el problema inverso debe resultar inviable para un adversario con recursos de cómputo realistas.

Esta dificultad se controla mediante el parámetro de seguridad λ , que en la práctica suele relacionarse con el tamaño de las claves expresado en bits. En RSA, este parámetro se identifica habitualmente con la longitud en bits del módulo público N . Sin embargo, una clave RSA de 2048 bits no proporciona 2048 bits de seguridad efectiva, sino un nivel de seguridad equivalente al coste de los mejores ataques conocidos de factorización. Por ello, las recomendaciones criptográficas distinguen entre longitud de clave y nivel de seguridad equivalente, o *security strength*, expresado también en bits [[Barker, 2020](#)].

En el caso de RSA, la seguridad del sistema se basa en la dificultad de factorizar un número entero grande obtenido como producto de dos números primos también grandes. Si denotamos estos primos por p y q , el módulo RSA se construye como

$$N = p \cdot q$$

A partir de estos valores se calcula también la función indicatriz de Euler, que para un entero positivo N cuenta cuántos enteros positivos menores o iguales que N son coprimos con él. Esta función es fundamental en RSA porque permite definir la relación modular entre el exponente público y el exponente privado [[Menezes et al., 1996](#)]. En el caso particular en que N es el producto de dos primos distintos p y q , se cumple que

$$\varphi(N) = (p - 1)(q - 1)$$

Después se elige un exponente público e , coprimo con $\varphi(N)$, y se calcula el exponente privado d como el inverso multiplicativo de e módulo $\varphi(N)$, de manera que

$$e \cdot d \equiv 1 \pmod{\varphi(N)}$$

De este modo, la clave pública p_k queda formada por el par (N, e) , mientras que la clave privada s_k contiene la información necesaria para conocer d , directamente o a través de la factorización del módulo N . En una versión simplificada del esquema, el cifrado de un mensaje m se realiza mediante la función de $Enc(m, p_k)$ definida como

$$c \equiv m^e \pmod{N}$$

y el descifrado se define como $Dec(c, s_k)$

$$m \equiv c^d \pmod{N}$$

En la Figura 5 describimos los pasos simplificados para la función $\text{KeyGen}(\lambda)$, en este caso el parámetro de seguridad es directamente el tamaño de módulo RSA N .

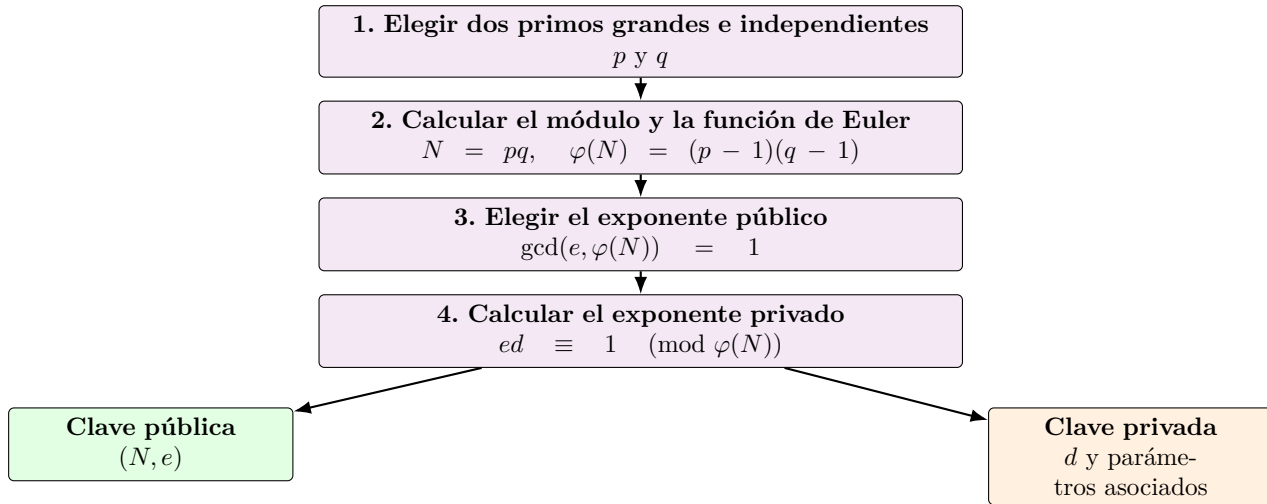


Figura 5: Proceso simplificado de generación de un par de claves RSA.

Las principales ventajas de RSA se apoyan, por tanto, en que multiplicar dos primos grandes es sencillo y eficiente, mientras que recuperar esos primos a partir únicamente de N es un problema computacionalmente muy costoso en condiciones normales para un adversario que utilice computación clásica. Esta dificultad práctica ha sido estudiada durante décadas mediante técnicas cada vez más avanzadas de factorización entera, como ilustra el *RSA Factoring Challenge*, propuesto por RSA Laboratories para incentivar la investigación sobre la factorización de grandes enteros semiprimos [Rivest et al., 1978; RSA Laboratories, 1991]. Por este motivo, RSA ha sido durante muchos años uno de los algoritmos más utilizados en certificados digitales y en infraestructuras de clave pública dentro de la Web.

Sin embargo, esta seguridad práctica no depende únicamente de la dificultad general del problema de factorización, sino que es importante que la implementación del sistema criptográfico se realice correctamente. Por ello, también depende de que los primos p y q se hayan generado correctamente, cumplan con la longitud recomendada de clave y, sobre todo, sean independientes de los utilizados por otras claves. Esta última idea será especialmente importante en este trabajo, ya que la presencia de factores compartidos entre módulos RSA supone una debilidad crítica que permite comprometer la seguridad del sistema sin necesidad de resolver el problema general de factorización.

2.2. Máximo común divisor y su relación con RSA

El máximo común divisor, o *Greatest Common Divisor* (GCD), de dos enteros es el mayor número que divide exactamente a ambos. Aunque se trata de un concepto básico de teoría de números, en el contexto de RSA adquiere una importancia especial: permite comprobar de forma eficiente si dos módulos tienen algún divisor común no trivial. Esta propiedad conecta directamente la generación incorrecta de claves con la posibilidad de factorizar módulos vulnerables.

Si consideramos dos módulos RSA distintos

$$N_1 = p \cdot q_1$$

y

$$N_2 = p \cdot q_2,$$

entonces ambos comparten el factor primo p . En ese caso, basta calcular

$$\gcd(N_1, N_2) = p$$

para recuperar directamente ese factor común. Una vez conocido, hallar q_1 y q_2 resulta inmediato mediante división y, con ello, también se puede reconstruir la información necesaria para comprometer las claves privadas asociadas, como se ha visto en la sección anterior. Por tanto, dos claves RSA que compartan un factor dejan de ser seguras de forma prácticamente instantánea. La Figura 6 resume esta situación, mostrando cómo la existencia de un factor primo común entre dos módulos RSA permite recuperarlo directamente mediante el cálculo del GCD.

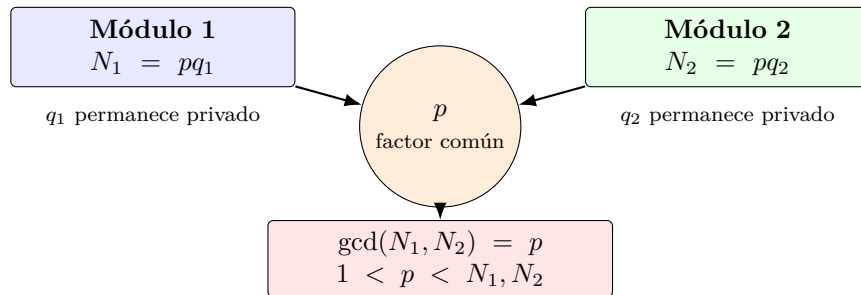


Figura 6: Ejemplo de dos módulos RSA que comparten un factor primo. El cálculo del GCD revela directamente dicho factor.

El interés del GCD en este trabajo no está solo en esta propiedad, sino también en que puede calcularse de forma muy eficiente mediante el algoritmo de Euclides. Este algoritmo se basa en una idea sencilla: el máximo común divisor de dos números no cambia si sustituimos el mayor por el resto de dividirlo entre el menor. Repitiendo este proceso de manera sucesiva se llega rápidamente a un resto cero, y el último resto no nulo es precisamente el máximo común divisor.

De forma simplificada, si $a > b$, el algoritmo aplica la relación

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

hasta alcanzar el caso final en el que el resto es cero. La eficiencia de este procedimiento es una de las razones por las que los ataques basados en GCD son tan potentes cuando existen factores compartidos entre claves RSA.

En el contexto de este trabajo, el máximo común divisor desempeña un papel central. Por un lado, constituye la herramienta matemática que permite explotar la debilidad cuando existen factores compartidos entre módulos RSA. Por otro, se convierte en la operación central de los algoritmos diseñados para detectar este problema a gran escala.

2.3. Entropía y generación de claves RSA

Aunque RSA es un esquema sólido desde el punto de vista matemático, su seguridad práctica depende de que la generación de claves se realice correctamente. En particular, para construir una clave RSA segura es necesario que los números primos p y q se generen de manera suficientemente aleatoria e independiente. Si esta condición no se cumple, pueden aparecer relaciones inesperadas entre distintas claves públicas, incluyendo la reutilización accidental de factores primos [Heninger et al.,

2012; Pelofske, 2024; Våge, 2022].

En este contexto, la entropía hace referencia al grado de imprevisibilidad disponible en el sistema para alimentar el proceso de generación aleatoria. En un entorno bien diseñado, esa entropía permite producir candidatos a primos que sean efectivamente impredecibles. Sin embargo, en sistemas reales esto no siempre ocurre. Una baja entropía durante la generación de primos puede hacer que distintas claves RSA compartan factores, lo que compromete completamente su seguridad.

Esta situación puede deberse a varias causas. Entre ellas se encuentran el uso incorrecto de librerías de generación aleatoria, errores de implementación o la falta de una fuente externa suficiente de entropía en el momento de generar la clave. En la práctica, la búsqueda de primos se apoya en algoritmos deterministas inicializados a partir de una semilla aleatoria; si esa semilla se repite o es predecible, distintos procesos pueden recorrer candidatos iguales o correlacionados y terminar generando primos repetidos. Existen recomendaciones específicas para evaluar las fuentes de entropía, como NIST Special Publication 800-90B [National Institute of Standards and Technology, 2001a]. Este problema resulta especialmente preocupante en dispositivos con recursos limitados, sistemas embebidos o equipos que generan claves en condiciones poco favorables, por ejemplo justo después del arranque, cuando el sistema aún puede no haber acumulado suficiente información impredecible procedente de eventos externos.

La Figura 7 resume esta cadena causal, desde una fuente de entropía insuficiente o un generador defectuoso hasta la aparición de módulos RSA vulnerables que pueden factorizarse mediante el cálculo del GCD.

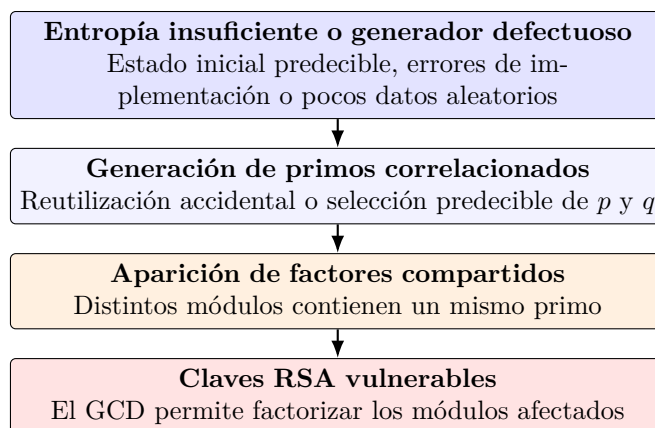


Figura 7: Cadena causal entre una generación aleatoria deficiente y la aparición de claves RSA vulnerables.

La tesis en la que se apoya este trabajo subraya que, en condiciones ideales, la probabilidad de que dos claves RSA distintas reutilicen un mismo primo dentro del enorme espacio de posibles primos de tamaño criptográfico es prácticamente nula [Våge, 2022]. Por eso, cuando en la práctica aparecen factores compartidos entre módulos públicos, no estamos ante una simple casualidad matemática, sino ante una señal muy clara de que ha existido un fallo en el proceso de generación.

Además, este problema no es meramente teórico. Trabajos previos ya mostraron que la baja entropía y otros errores de implementación podían dar lugar a claves públicas vulnerables en sistemas reales, incluyendo dispositivos conectados a Internet [Heninger et al., 2012]. Esta línea de investigación se ha aplicado a certificados publicados en *CT logs* y también ha motivado algoritmos más eficientes para analizar estas situaciones a gran escala [Pelofske, 2024; Våge, 2022].

Por tanto, la entropía es importante en este trabajo porque ayuda a explicar el origen de la vulnerabilidad que se quiere estudiar. En la práctica no es sencillo comprobar equipo por equipo si las

claves se han generado en buenas condiciones, especialmente cuando intervienen muchos certificados y dispositivos conectados a Internet. Por eso, una forma razonable de estudiar el problema es analizar grandes conjuntos de claves públicas ya emitidas y buscar en ellas factores compartidos. Si la generación de claves no se realiza siempre en condiciones ideales, entonces la auditoría empírica de la infraestructura criptográfica desplegada en Internet se convierte en una tarea especialmente relevante desde el punto de vista de la ciberseguridad.

2.4. Certificate Transparency logs

Para poder estudiar este problema en sistemas reales es necesario contar con una fuente de claves públicas que contenga un volumen de datos suficiente y que sea accesible. En este trabajo esa fuente serán los *Certificate Transparency logs*, o *CT logs*, que almacenan certificados digitales emitidos para dominios y servicios de Internet.

Antes de explicar su utilidad, conviene recordar brevemente qué papel desempeñan las claves y los certificados digitales en la Web. Cada servidor conserva una clave privada que debe mantenerse secreta e idealmente no debe abandonar el sistema. La clave pública correspondiente a esa clave privada es necesaria para que otras partes puedan verificar operaciones criptográficas asociadas, por ejemplo mediante mecanismos de firma digital. Un certificado digital, normalmente en formato X.509, vincula esa clave pública con una identidad o dominio. Una autoridad certificadora comprueba dicha vinculación y firma el certificado con su propia clave privada; el cliente puede verificar la firma mediante la clave pública de la autoridad, directamente o a través de una cadena de confianza.

El formato X.509 funciona como estándar de facto para el manejo de certificados en infraestructuras de clave pública. Fue propuesto inicialmente como parte de la serie X.500 de la Unión Internacional de Telecomunicaciones (ITU, por sus siglas en inglés) para la interconexión de sistemas abiertos [Union, 1988]. De forma simplificada, un certificado X.509 contiene información estructurada como la versión, el número de serie, el identificador del algoritmo de firma, el emisor, el periodo de validez, el sujeto o titular del certificado, la información de clave pública del sujeto y la firma digital del certificado.

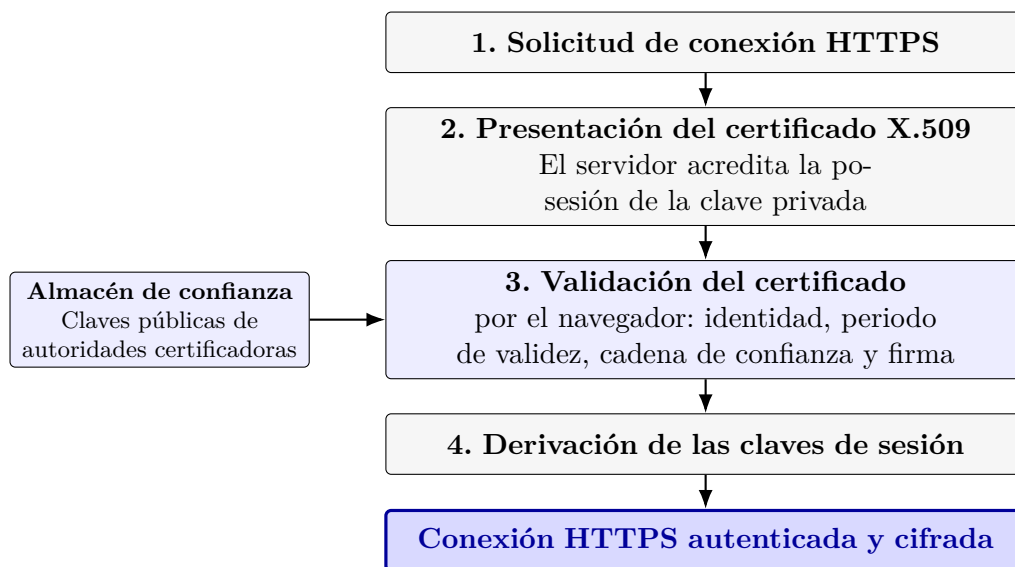


Figura 8: Esquema simplificado del establecimiento de una conexión TLS. Elaboración propia a partir de RFC 8446.

En la práctica, este mecanismo es uno de los pilares de TLS, el protocolo utilizado para proteger HTTPS, introducido en el Capítulo 1. Como resume la Figura 8, el cliente inicia la negociación, el

servidor presenta su certificado y demuestra que posee la clave privada asociada, y ambas partes establecen claves de sesión para proteger el tráfico posterior [Rescorla, 2018]. La clave privada del servidor no se transmite; lo que se publica mediante el certificado es su clave pública. Los certificados digitales hacen así posible la autenticación del servidor y, al mismo tiempo, exponen claves públicas. Estas claves pueden analizarse con fines de auditoría criptográfica.

En este contexto surge *Certificate Transparency*, un marco diseñado para aportar mayor transparencia al ecosistema de certificación web. Su idea principal es que los certificados emitidos por las autoridades certificadoras queden registrados en logs públicos, verificables y de solo anexo, de manera que su emisión pueda ser observada y supervisada por terceros [Laurie et al., 2021]. Esto dificulta que una autoridad certificadora emita certificados incorrectos o fraudulentos sin que nadie pueda detectarlo.

Esto conecta de forma directa con la línea seguida en este TFM. Gracias a los *CT logs*, es posible pasar de un análisis puramente teórico sobre debilidades de RSA a una auditoría empírica sobre claves públicas realmente desplegadas en Internet. Como veremos en el capítulo siguiente, esta posibilidad ha dado lugar a trabajos previos relevantes y justifica la necesidad de seguir buscando métodos más eficientes para detectar factores compartidos a gran escala.

Capítulo 3

Estado del arte

Tras haber presentado los conceptos necesarios para entender este trabajo, en este capítulo revisaremos los principales antecedentes relacionados con el problema. En particular, nos centraremos en cómo la literatura ha estudiado las debilidades en la generación de claves RSA, la detección de factores compartidos entre claves públicas y el análisis de este tipo de vulnerabilidades sobre grandes conjuntos de datos reales. Además, revisaremos los enfoques computacionales clásicos utilizados hasta ahora, sus principales limitaciones y la propuesta más reciente en la que se apoya este trabajo, con el fin de situar nuestra aportación a la literatura previa.

3.1. Detección de factores compartidos en claves públicas RSA

Como ya hemos explicado en el capítulo anterior, la seguridad práctica de RSA depende en gran medida de que la generación de claves se realice correctamente. Por ello, una de las líneas más importantes dentro de la literatura ha consistido en analizar hasta qué punto problemas de baja entropía o fallos de implementación pueden dar lugar a claves vulnerables en sistemas reales. Uno de los trabajos más influyentes en esta dirección mostró que la existencia de claves débiles no era una posibilidad puramente teórica, sino un problema que podía observarse en dispositivos conectados a Internet [Heninger et al., 2012]. En particular, este trabajo mostró la presencia de claves repetidas y de módulos RSA que compartían factores primos, lo que permitía asociar muchas de estas debilidades a problemas prácticos en la generación de claves. Este resultado es importante porque demuestra que la seguridad de RSA no depende solo de su base matemática, sino también de que las claves se generen en buenas condiciones.

Posteriormente, Pelofske [2024] profundiza en el estudio de la fase de creación de los primos. Pelofske señala que estos fallos pueden deberse a un uso incorrecto de librerías de generación aleatoria o a la ausencia de una fuente externa de entropía suficiente en el momento de generar la clave. Våge [2022] recuerda que, en condiciones ideales, la probabilidad de que dos claves RSA distintas reutilicen un mismo primo dentro del enorme espacio de posibles primos criptográficos es prácticamente nula. Desde el punto de vista del estado del arte, esto resulta especialmente importante porque justifica toda una línea de trabajos orientados a auditar claves públicas reales en busca de este tipo de debilidades. En otras palabras, en estudiar si las implementaciones reales respetan las condiciones necesarias para que esa seguridad teórica se mantenga en la práctica.

En este sentido, tanto la tesis como el artículo de los que partimos siguen la idea de que la baja entropía y los errores de implementación pueden dar lugar a factores compartidos entre módulos RSA, que constituyen una vulnerabilidad grave que merece ser analizada empíricamente a gran escala [Pelofske, 2024; Våge, 2022]. A partir de aquí, el siguiente paso natural dentro de la literatura fue centrarse en cómo detectar de forma eficiente esas claves vulnerables. En este contexto, la presencia de factores compartidos entre módulos públicos se convirtió rápidamente en un objetivo de estudio especialmente relevante, ya que permite identificar claves vulnerables de manera directa. Por ello, la

cuestión dejó de ser únicamente si este fenómeno podía ocurrir y pasó a centrarse en cómo encontrar esos casos dentro de grandes colecciones de claves públicas reales.

Tal y como plantea Pelofske [2024], el verdadero problema no está en calcular el máximo común divisor entre dos módulos concretos, ya que esa operación es eficiente, sino en que no se conoce de antemano qué pares de claves podrían compartir factores. Esto hace que una búsqueda exhaustiva basada en comparar todos los pares posibles se vuelva rápidamente inviable cuando el número de módulos crece. En conjuntos grandes de claves públicas, el número de comparaciones necesarias aumenta de forma cuadrática al compararse todos los pares posibles, lo que convierte el enfoque de fuerza bruta en una opción poco realista para auditorías a gran escala cuando el número de certificados a comprobar crece.

Precisamente por esta razón, la literatura comenzó a desarrollar métodos computacionales específicos para realizar este tipo de análisis de manera más eficiente. En lugar de tratar cada comparación de forma aislada, estos enfoques buscan explotar la estructura global del conjunto de módulos RSA para detectar divisores comunes no triviales con un coste mucho menor que el de una enumeración completa de todos los pares. Un enfoque *batch GCD* se aplicó a grandes colecciones de claves RSA extraídas de *CT logs* [Våge, 2022]. Sobre el mismo problema de detección masiva, se propuso después una alternativa más eficiente al método clásico [Pelofske, 2024]. Esta es la idea general que dará lugar a los distintos enfoques de *batch GCD*, que más adelante explicaremos con mayor detalle.

Así, dentro del estado del arte, la detección de factores compartidos en claves públicas RSA debe entenderse no solo como una consecuencia natural de las debilidades en la generación de claves, sino también como un problema computacional en sí mismo por el tamaño del espacio de búsqueda. Es precisamente esa doble dimensión, criptográfica por un lado y algorítmica por otro, la que explica el interés que ha despertado este tema en la literatura reciente.

A partir de ahí, el siguiente paso fue estudiar cómo analizar conjuntos de datos cada vez mayores de forma eficiente desde el punto de vista computacional, con el objetivo de comprobar hasta qué punto estas debilidades aparecían realmente en sistemas desplegados en Internet.

3.2. Análisis masivo de claves débiles en Internet

Una vez planteado el problema de detectar factores compartidos entre claves públicas RSA, la siguiente evolución natural dentro de la literatura fue trasladar este análisis a conjuntos de datos cada vez mayores. El interés dejó de centrarse únicamente en demostrar que la vulnerabilidad era posible y pasó a orientarse hacia auditorías criptográficas a gran escala, con el objetivo de comprobar hasta qué punto estas debilidades aparecían realmente en sistemas desplegados en Internet y poder aplicar medidas como la generación de nuevas claves o la revocación de certificados.

En este contexto, uno de los trabajos más relevantes realizó un escaneo masivo de servicios accesibles en Internet y recopiló millones de claves públicas RSA procedentes principalmente de HTTPS y SSH [Heninger et al., 2012]. Sus resultados mostraron que era posible factorizar un número significativo de estas claves, lo que confirmó que el problema no era anecdótico y que existían implementaciones reales vulnerables debido a deficiencias en la generación de claves.

A partir de ahí, otros trabajos ampliaron tanto el volumen analizado como el número de claves factorizadas. Hastings et al. analizaron 81 millones de claves y factorizaron más de 313.000 [Hastings et al., 2016], mientras que Kilgallin y Vasko analizaron unos 57 millones de claves procedentes de escaneos de Internet y lograron factorizar 249.548 [Kilgallin y Vasko, 2019], tal y como resume también la tesis de Våge [Våge, 2022].

Estos resultados muestran que la búsqueda de factores compartidos dejó de ser rápidamente un problema puramente académico para convertirse en una herramienta útil de auditoría sobre infraestructuras reales. Además, la literatura comenzó a señalar que el auge del Internet de las Cosas podía estar relacionado con la persistencia de este tipo de vulnerabilidades, ya que muchos de estos dispositivos son ligeros, disponen de pocos recursos y pueden contar con fuentes de entropía pobres. Esto hace que el problema sea especialmente relevante en un contexto con un número creciente de dispositivos conectados.

Por otro lado, el origen de las claves analizadas también resultó ser un aspecto importante. Våge destaca una diferencia relevante entre las claves obtenidas mediante escaneos de Internet y las procedentes de certificados publicados en *Certificate Transparency logs*. Mientras que los escaneos pueden devolver certificados autofirmados, de prueba o de uso interno, las claves presentes en *CT logs* están asociadas a certificados firmados por autoridades certificadoras reconocidas por los principales navegadores. Esto hace que los *CT logs* resulten especialmente interesantes cuando el objetivo es estudiar la criptografía desplegada en el ecosistema web público.

La tesis que sirve de base a este TFM se sitúa precisamente en esta línea. Frente a trabajos anteriores más centrados en escaneos de Internet, Våge recopiló 159.377.664 claves RSA distintas de 2048 bits extraídas de *CT logs*, el mayor conjunto de este tipo utilizado hasta ese momento. En la ejecución general del algoritmo se encontraron ocho claves vulnerables, todas ellas emitidas por ZeroSSL, y un análisis posterior centrado en esta autoridad certificadora permitió factorizar 355 claves únicas dentro de un conjunto de más de 700.000 [Våge, 2022].

La conclusión de la tesis resulta especialmente relevante para nuestro trabajo: tanto ese estudio como el de Kilgallin y Vasko [2019] apenas habían analizado una pequeña fracción del volumen total de certificados subidos a los *CT logs* desde 2013. Esto pone de manifiesto que el problema seguía lejos de estar agotado y que todavía existía margen para investigaciones más grandes y exhaustivas. Precisamente en ese punto cobra importancia la necesidad de contar con algoritmos cada vez más eficientes para detectar factores compartidos a gran escala.

3.3. Enfoque clásico basado en product tree y remainder tree

Para que este tipo de análisis masivo sea viable, la literatura tuvo que ir más allá de la comparación entre todos los pares posibles de módulos RSA. Como ya se ha comentado, una estrategia basada en calcular el GCD de cada par crece de forma cuadrática y se vuelve poco realista cuando el número de claves es muy grande. Por ello, se desarrollaron algoritmos específicos capaces de explotar la estructura del conjunto completo de módulos para reducir de forma importante el coste del análisis.

Dentro de estos enfoques, el método clásico que ha acabado consolidándose es el basado en la construcción de un *product tree* seguido de un *remainder tree*. Este método constituye la referencia principal frente a la que se comparan las propuestas más recientes [Pelofske, 2024; Våge, 2022].

La idea general de este enfoque consiste en multiplicar de forma jerárquica todos los módulos RSA del conjunto para construir un árbol de productos. Después, a partir de esa estructura, se calcula un árbol de residuos que permite determinar, en las hojas, si un módulo comparte algún factor con el producto del resto [Pelofske, 2024]. De esta forma, no es necesario comparar cada clave con todas las demás de manera independiente, sino que el problema se reorganiza mediante operaciones sobre árboles que aprovechan mejor la estructura global del conjunto.

La tesis de Våge explica además cómo este método puede adaptarse a ejecuciones de gran escala dividiendo la colección completa de módulos en varios lotes o *batches* [Våge, 2022]. Para cada lote se construye su correspondiente *product tree* y *remainder tree*, y posteriormente se cruzan los distintos

bloques para detectar factores compartidos entre lotes diferentes. Este planteamiento permite controlar mejor el tamaño de los enteros intermedios y hacer viable el análisis de conjuntos mucho mayores de los que podría manejar una ejecución monolítica en memoria.

Por tanto, el enfoque clásico basado en *product tree* y *remainder tree* supuso un avance importante dentro del estado del arte, ya que permitió pasar de estrategias conceptualmente simples pero inasumibles a procedimientos realmente utilizables para auditar grandes colecciones de claves públicas RSA. Los estudios empíricos comentados en el apartado anterior se basan en esta técnica; sin esta familia de métodos, gran parte de esos análisis no habría sido posible.

Aunque este método supuso un gran avance respecto a la comparación ingenua de todos los pares, la propia literatura ha puesto de manifiesto que su aplicación práctica a gran escala sigue presentando dificultades importantes. De hecho, que este enfoque haya sido el dominante durante años no significa que esté exento de limitaciones. Cuando se pretende seguir aumentando el tamaño de los conjuntos analizados, empiezan a aparecer dificultades prácticas relacionadas con el tiempo de ejecución, el consumo de memoria y la gestión de enteros de gran tamaño. Estas limitaciones no invalidan el método, pero sí explican por qué resulta razonable seguir buscando alternativas más eficientes.

Una de las principales dificultades aparece en la parte alta de los árboles, donde los enteros intermedios alcanzan tamaños enormes. La tesis de Våge señala que el cuello de botella del algoritmo se encuentra precisamente en estas etapas, ya que el coste computacional crece a medida que se opera con números cada vez mayores. Esto no solo afecta al tiempo de ejecución, sino también a la cantidad de memoria necesaria para mantener la estructura del cálculo.

Además, cuando el análisis se lleva a conjuntos realmente grandes, entran en juego problemas más prácticos de ingeniería asociados al coste computacional. En la ejecución descrita en la tesis, el experimento principal tuvo que dividirse en 13 lotes de alrededor de 12 millones de claves cada uno, y el proceso completo requirió del orden de 180 GB de memoria RAM, aproximadamente 1 TB de almacenamiento en disco y unas 88 horas de ejecución utilizando 30 núcleos [Våge, 2022]. Estas cifras dejan claro que, aunque el método clásico funciona, escalarlo no resulta trivial y exige recursos considerables.

Esta tesis también menciona otras complicaciones asociadas a este tipo de ejecuciones, como la necesidad de almacenar los datos de forma sistemática o los problemas derivados de trabajar con enteros muy grandes en bibliotecas de aritmética multiprecisión. Todo ello refuerza la idea de que, cuando el objetivo es seguir ampliando el análisis sobre volúmenes masivos de certificados, el algoritmo utilizado pasa a ser un factor decisivo.

En la propia conclusión del trabajo, Våge señala que una mejora del algoritmo *batch GCD* permitiría investigaciones más grandes y más completas sobre el estado de la criptografía en la práctica. Esta observación resulta especialmente importante, porque enlaza de forma directa con la propuesta de Pelofske y con la motivación de este TFM. Por ello, las limitaciones del método clásico no deben entenderse como un fracaso del enfoque, sino como la consecuencia natural de intentar escalar el análisis a conjuntos cada vez mayores. Precisamente por eso, el siguiente paso dentro del estado del arte consiste en estudiar propuestas que mantengan la utilidad del *batch GCD*, pero reduzcan parte del coste computacional asociado al método tradicional.

3.4. Binary Tree Batch GCD como propuesta reciente

En este contexto se propone una alternativa al enfoque clásico denominada *Binary Tree Batch GCD* [Pelofske, 2024]. Su punto de partida es el mismo problema descrito anteriormente: cómo realizar un ataque *all-to-all GCD* sobre grandes conjuntos de módulos RSA para detectar factores compartidos debidos a baja entropía o a fallos de implementación.

Desde la perspectiva del estado del arte, lo más relevante de esta propuesta es que busca reducir parte del coste práctico asociado al método tradicional basado en *product tree* y *remainder tree*. El nuevo enfoque evita construir explícitamente el *remainder tree* y mantiene una complejidad asintótica muy similar a la del método clásico.

Además, en los experimentos presentados por Pelofske, el algoritmo obtiene mejores tiempos de ejecución que el enfoque tradicional, con una mejora práctica media aproximada de seis veces [Pelofske, 2024]. Esto resulta especialmente interesante en un contexto como el nuestro, donde el problema no es solo detectar factores compartidos, sino hacerlo sobre conjuntos de datos cada vez mayores.

Por tanto, dentro de la literatura reciente, el *Binary Tree Batch GCD* puede interpretarse como una evolución natural del enfoque clásico. No cambia el problema que se quiere resolver, pero sí propone una forma distinta de abordarlo, con el objetivo de mejorar el rendimiento práctico y facilitar análisis de mayor escala sobre claves RSA potencialmente vulnerables.

3.5. Encaje y motivación del presente trabajo

A la vista de lo revisado en este capítulo, podemos situar la aportación de este Trabajo de Fin de Máster dentro del estado del arte actual.

Por un lado, la tesis de Våge [2022] demostró que los *Certificate Transparency logs* constituyen una fuente valiosa para estudiar factores compartidos en claves RSA reales y que este tipo de análisis sigue ofreciendo resultados relevantes en la práctica. Además, dejó abierta una idea especialmente importante para nuestro trabajo: si el algoritmo *batch GCD* pudiera mejorarse, sería posible llevar a cabo investigaciones más grandes y más completas sobre el estado real de la criptografía desplegada en Internet.

Por otro lado, el artículo de Pelofske [2024] propone precisamente una mejora algorítmica orientada a ese mismo problema, mostrando que el *Binary Tree Batch GCD* puede ofrecer una ventaja práctica significativa frente al método clásico basado en *remainder tree*. Esto hace que ambas referencias encajen de forma especialmente natural entre sí: la tesis aporta el contexto aplicado, la fuente de datos y la motivación experimental, mientras que el artículo aporta la propuesta algorítmica más reciente.

Nuestro TFM se sitúa precisamente en la intersección entre ambos trabajos. Partiremos del problema y del contexto experimental planteados en la tesis, centrados en el análisis de claves RSA extraídas de *CT logs*, pero sustituiremos el núcleo algorítmico empleado en dicho trabajo por la propuesta de Pelofske. Con ello, buscaremos comprobar si esta alternativa constituye una mejora real en términos prácticos y si puede servir como base para futuras auditorías sobre conjuntos aún mayores de certificados.

De este modo, la aportación de este trabajo no consistirá en replantear el problema desde cero, sino en continuar una línea de investigación ya abierta, incorporando una herramienta más reciente y potencialmente más eficiente. Este posicionamiento resulta, a nuestro juicio, especialmente adecuado para un Trabajo de Fin de Máster, ya que combina revisión crítica de trabajos previos, implementación experimental y análisis aplicado sobre un problema real de ciberseguridad.

Una vez establecido este marco, en el capítulo siguiente nos centraremos ya en la parte metodológica del trabajo. En particular, describiremos con más detalle el fundamento del *batch GCD*, el algoritmo *Binary Tree Batch GCD* y las decisiones de implementación adoptadas para llevar a cabo nuestros experimentos.

Capítulo 4

Metodología y algoritmo propuesto

Una vez revisados los antecedentes principales del problema dentro de la literatura previa, en este capítulo pasamos a describir la metodología seguida en nuestro trabajo.

Como se ha visto en el capítulo anterior, partimos de una línea de investigación ya existente centrada en la detección de factores compartidos en claves RSA públicas obtenidas a partir de *Certificate Transparency logs*. En particular, tomamos como referencia un planteamiento experimental previo, pero sustituimos el enfoque clásico basado en *product tree* y *remainder tree* por el algoritmo más reciente *Binary Tree Batch GCD* de Pelofske [2024].

Nuestro objetivo es estudiar si esta alternativa resulta adecuada para el problema que nos ocupa y si puede constituir una mejora práctica frente al método tradicional. Para ello, no nos limitamos a revisar el algoritmo desde un punto de vista teórico, sino que lo implementamos y lo aplicamos en dos contextos distintos: primero sobre datos sintéticos generados de forma controlada y, después, sobre datos reales obtenidos a partir de *CT logs*.

La metodología del trabajo se apoya, por tanto, en las siguientes fases. En primer lugar, se estudia el fundamento general de los métodos *batch GCD*, ya que constituyen la base matemática y algorítmica de este tipo de ataques. En segundo lugar, se analiza el algoritmo *Binary Tree Batch GCD* y su aplicación al problema de detección de factores compartidos. Y, en tercer lugar, se desarrolla una implementación práctica eficiente desde el punto de vista computacional y se evalúa su comportamiento en escenarios sintéticos y reales.

A continuación, describimos primero el fundamento general del *batch GCD*, después el funcionamiento del algoritmo seleccionado y, por último, la implementación desarrollada para llevar a cabo nuestros experimentos.

4.1. Herramientas utilizadas

Antes de describir el algoritmo y la implementación, resumimos las herramientas empleadas en el trabajo y las razones de su elección.

El lenguaje principal de la implementación es **C++17**. Esta elección responde a la necesidad de un control fino sobre la gestión de memoria, fundamental en un problema donde los enteros intermedios alcanzan tamaños muy grandes, y a la posibilidad de enlazar directamente con bibliotecas nativas optimizadas. Para la aritmética de precisión arbitraria se utiliza la **GNU Multiple Precision Arithmetic Library (GMP)** [Granlund y the GMP Development Team, 2023], referencia habitual en computación con enteros grandes por sus algoritmos de multiplicación y cálculo de máximo común divisor altamente optimizados. La paralelización opcional de la fase de enumeración final se apoya en **OpenMP** [OpenMP Architecture Review Board, 2021].

Como implementaciones de referencia se emplean las versiones **Python** originales del repositorio ofrecido por Pelofske [2024], ejecutadas con Python 3.12 y las bibliotecas NumPy, pandas, psutil y PyCryptodome. Mantener estas implementaciones sin modificar su lógica permite atribuir las diferencias de rendimiento al algoritmo y al entorno de ejecución, no a cambios en el código de referencia.

Para la parte de datos reales se utilizan **OpenSSL**, con el que se extraen el módulo RSA y el emisor de cada certificado X.509, y el repositorio público *ct-archive* [Geomys, 2026] como fuente de volcados de *Certificate Transparency logs*. La orquestación del flujo de descarga, extracción y consolidación se realiza mediante scripts y cuadernos que se conservan como material complementario del trabajo. Las versiones concretas de todo el entorno se recogen en la Tabla 1 del capítulo de resultados.

4.2. Fundamento del *batch GCD*

Como ya se ha visto en los capítulos anteriores, la detección de factores compartidos entre claves públicas RSA se basa en una idea sencilla: si dos módulos comparten un factor primo, dicho factor puede recuperarse calculando su máximo común divisor. El problema aparece cuando no queremos estudiar solo un par de módulos, sino un conjunto muy grande de ellos. En ese escenario, ya no basta con saber que el cálculo individual de un *gcd* es eficiente, sino que debemos enfrentarnos al hecho de que no conocemos de antemano qué pares de claves podrían compartir factores.

Si el conjunto contiene M módulos RSA, una estrategia de fuerza bruta exigiría realizar

$$\frac{M(M-1)}{2}$$

comparaciones distintas, ya que habría que estudiar todos los pares posibles. Aunque cada cálculo individual de *gcd* sea barato, el número total de operaciones crece de forma cuadrática con el tamaño del conjunto, lo que convierte este enfoque en una opción poco realista cuando trabajamos con miles, millones o incluso cientos de millones de claves. El artículo de Pelofske insiste precisamente en esta idea al presentar el problema como un ataque *all-to-all GCD* [Pelofske, 2024].

A partir de esta limitación surge la necesidad de diseñar métodos que detecten factores compartidos sin comparar cada módulo con todos los demás. Esa es la idea de los algoritmos *batch GCD*: reorganizar el cálculo para aprovechar productos agregados del conjunto y reducir el coste práctico.

Dentro de esta familia, el método clásico descrito en la tesis de Våge parte de un *product tree* y un *remainder tree*. Primero agrupa los módulos de forma jerárquica y forma productos intermedios; después usa el árbol de residuos para comprobar en las hojas si un módulo comparte algún factor con el producto del resto. Así, el análisis deja de ser una colección de comparaciones independientes y pasa a organizarse mediante operaciones sobre árboles.

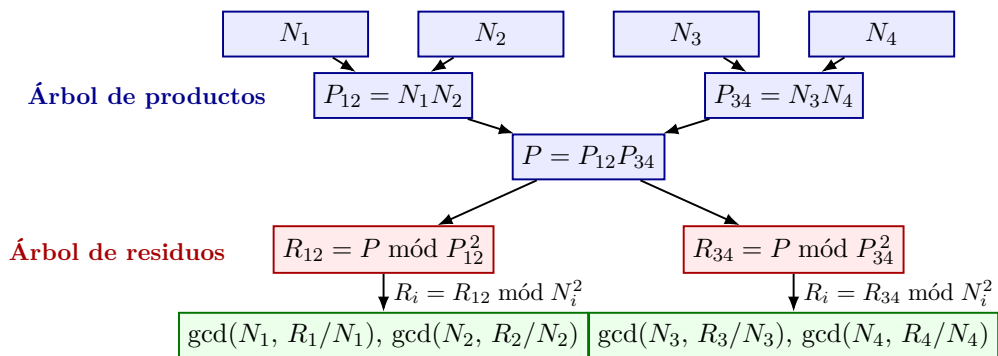


Figura 9: Esquema del método clásico. Azul: productos RSA; rojo: residuos; verde: GCD finales.

La Figura 9 resume el recorrido del método clásico. Los módulos se combinan primero hasta obtener el producto global P . A continuación, los residuos se propagan en sentido descendente y permiten calcular para cada hoja el máximo común divisor con el producto de los demás módulos, sin enumerar explícitamente todos los pares.

La tesis explica además que, para trabajar con colecciones muy grandes de claves, este planteamiento puede extenderse al denominado *batch GCD algorithm*. En este caso, el conjunto completo de claves se divide en varios lotes o *batches*, y sobre cada uno de ellos se construyen sus correspondientes árboles de productos y residuos. Este paso permite controlar mejor el tamaño de los enteros en la parte alta de los árboles y, por tanto, reducir el cuello de botella asociado al manejo de productos gigantescos. Sin embargo, esta división introduce también una consecuencia importante: para detectar factores compartidos entre módulos pertenecientes a lotes distintos, cada producto grande de un lote debe comprobarse frente a los árboles de residuos de los demás, de modo que el algoritmo debe repetirse múltiples veces entre lotes [Våge, 2022].

Por tanto, desde un punto de vista metodológico, el fundamento del *batch GCD* puede resumirse en la siguiente idea: en lugar de preguntar por separado si cada par de módulos comparte un factor, se reorganiza el problema para explotar productos agregados que contienen información sobre muchos módulos a la vez. Esto permite detectar divisores comunes sin recorrer explícitamente todas las comparaciones posibles. En otras palabras, el *batch GCD* no elimina la lógica del ataque por máximo común divisor, pero sí sustituye una búsqueda par a par por una estrategia global mucho más escalable.

Pelofske [2024] explica que el objetivo de fondo consiste en cubrir, de manera eficiente, el equivalente a todas las comparaciones necesarias en un problema *all-to-all GCD*. Según el artículo, una propiedad importante del máximo común divisor es que, si dividimos el conjunto de enteros en dos mitades y calculamos el *gcd* entre los productos de ambas, obtenemos de una sola vez información sobre muchos divisores comunes que aparecerían en comparaciones individuales. Aunque una sola partición no cubre todas las combinaciones posibles, la idea puede aplicarse de forma recursiva mediante una estructura arbórea, lo que permite recuperar la información necesaria sobre todos los factores compartidos presentes en el conjunto.

Visto así, el *batch GCD* no debe entenderse únicamente como una optimización práctica, sino como el núcleo algorítmico que hace viable este tipo de auditorías criptográficas a gran escala. Sin una reorganización de este tipo, analizar colecciones masivas de claves RSA reales, como las procedentes de *Certificate Transparency logs*, sería computacionalmente muy costoso. Por ello, tanto el enfoque clásico basado en *product tree* y *remainder tree* como la propuesta más reciente de Pelofske pueden interpretarse como distintas formas de materializar una misma idea general: transformar un ataque basado en comparaciones par a par en un procedimiento global y escalable.

Una vez establecido este fundamento general, en el siguiente apartado describiremos ya con más detalle el algoritmo *Binary Tree Batch GCD*, que presenta una mejora significativa sobre este enfoque clásico basado en *product tree* y *remainder tree*.

4.3. Algoritmo *Binary Tree Batch GCD*

Una vez introducido el fundamento general de los métodos *batch GCD*, en este apartado describiremos con más detalle el algoritmo *Binary Tree Batch GCD*, que constituye el núcleo metodológico de nuestro trabajo. Aquí nos centraremos en su funcionamiento y en la lógica que seguiremos posteriormente en nuestra implementación.

El algoritmo consta de tres pasos principales. En primer lugar, se construye un árbol binario de productos a partir del conjunto de módulos RSA de entrada. En segundo lugar, se toma el producto

de todos los valores de gcd obtenidos durante la construcción del árbol y se agrupan en un único entero, que el artículo denota por B . En tercer lugar, se enumera el conjunto original de módulos y se calcula el gcd entre cada uno de ellos y B , de modo que los resultados no triviales permiten recuperar los factores compartidos [Pelofske, 2024].

La Figura 10 resume la secuencia completa: los productos avanzan por el árbol, los máximos comunes divisores se acumulan en B y, al alcanzar la raíz, se comprueba cada módulo de entrada.

```

Entrada: modulos RSA N[1..M]
nivel <- N
B <- 1
mientras tamano(nivel) > 1:
    siguiente <- []
    para cada pareja (A, C) de nivel:
        B <- B * gcd(A, C)
        siguiente.anadir(A * C)
    si queda un nodo sin pareja:
        siguiente.anadir(nodo)
    nivel <- siguiente
para cada modulo N_i:
    factor <- gcd(N_i, B)
    si 1 < factor < N_i:
        informar(N_i, factor)

```

Figura 10: Pseudocódigo simplificado del algoritmo *Binary Tree Batch GCD*. El esquema muestra la construcción del árbol de productos, la acumulación de los valores de GCD en B y la enumeración final de los módulos de entrada.

Sea $N = \{N_1, N_2, \dots, N_M\}$ el conjunto de módulos de entrada. En el nivel inferior se agrupan por parejas y, para cada par (A, C) , se calculan simultáneamente $gcd(A, C)$ y el producto $A \cdot C$. El máximo común divisor se incorpora a B , mientras que el producto forma el nodo padre. La misma operación se repite sobre los productos de cada nivel hasta alcanzar la raíz; si un nivel contiene un número impar de nodos, el elemento sin pareja se propaga al siguiente sin modificar.

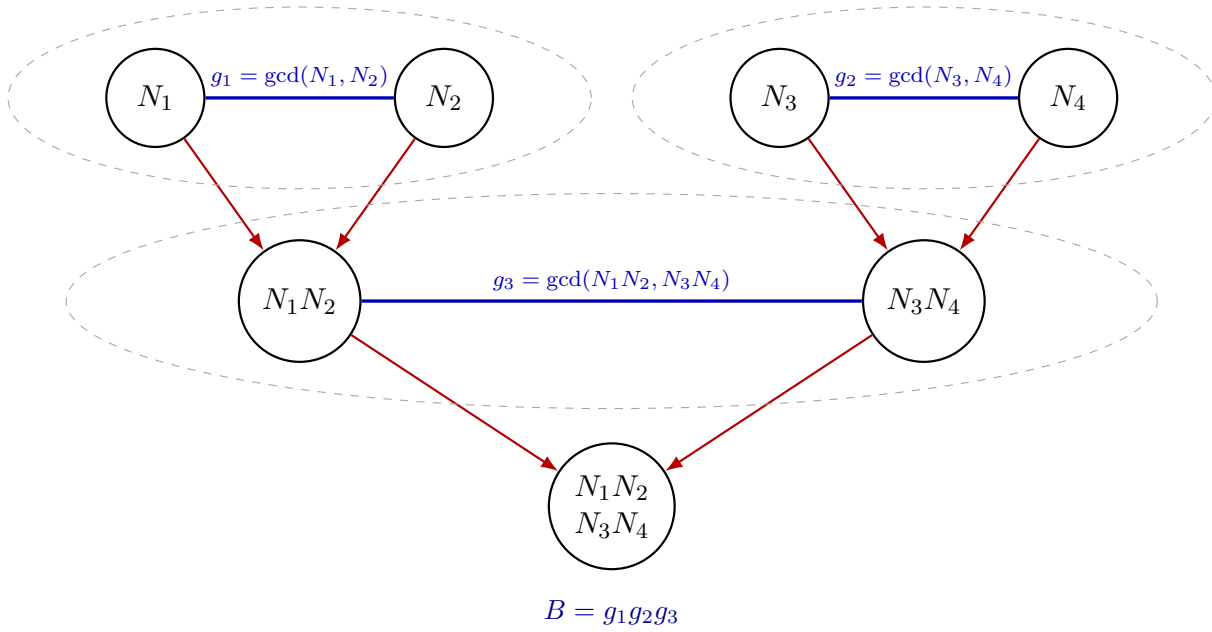


Figura 11: Construcción del árbol en *Binary Tree Batch GCD*. En cada agrupación se calcula el producto que asciende al siguiente nivel y el GCD entre las dos ramas, cuyos valores no triviales se agregan en B . Las líneas rojas representan la construcción de los productos y las líneas azules los cálculos de GCD entre ramas hermanas.

La Figura 11 muestra el caso simplificado de cuatro módulos. A diferencia del método clásico, el recorrido termina al alcanzar la raíz y no necesita construir después un árbol de residuos. La intuición detrás de este diseño es importante. Si dividimos el conjunto de enteros en dos mitades y calculamos el gcd entre el producto de una mitad y el producto de la otra, obtenemos de una sola vez información equivalente a un gran número de comparaciones individuales entre elementos de ambas mitades. Una sola partición no cubre todas las combinaciones posibles, pero al aplicar esta idea recursivamente sobre un árbol binario sí se consigue cubrir, en conjunto, toda la información necesaria para un problema *all-to-all GCD*.

Una vez construido el árbol, el algoritmo dispone de todos los valores de gcd generados en sus nodos internos. Si el árbol tiene M hojas, el número de nodos internos es $M - 1$, y por tanto se obtiene exactamente ese número de resultados intermedios. El siguiente paso consiste en multiplicarlos entre sí para formar un único entero, que denotaremos por B . Formalmente, si llamamos $g_1, g_2, \dots, g_{M-1}$ a los valores de gcd obtenidos durante la construcción del árbol, entonces se define

$$B = \prod_{j=1}^{M-1} g_j$$

Si $B = 1$, entonces no se ha encontrado ningún divisor común no trivial en el conjunto de módulos analizado. En cambio, si $B > 1$, ese entero contiene la información agregada sobre los factores compartidos presentes en el conjunto.

El tercer y último paso consiste en recorrer de nuevo el conjunto original de módulos y calcular, para cada N_i , el valor

$$\gcd(N_i, B)$$

Cualquier resultado no trivial obtenido en esta enumeración indica que el módulo N_i comparte algún factor con otro módulo del conjunto. De este modo, el algoritmo transforma toda la información agregada en B en factores concretos asociados a módulos individuales.

Por tanto, el número exacto de operaciones de gcd realizadas por este procedimiento es $M - 1$ durante la construcción del árbol, más M en la fase final de enumeración, es decir, un total de

$$2M - 1$$

operaciones de gcd [Pelofske, 2024]. Esto contrasta fuertemente con el número cuadrático de comparaciones que exigiría una estrategia basada en revisar todos los pares posibles.

Por ello, una ventaja importante de este enfoque es que no necesita construir explícitamente un *remainder tree*, como ocurre en el método clásico. En su lugar, la información relevante se va acumulando a través de los gcd calculados en los nodos del árbol y del producto agregado B . Esta es precisamente la característica que permite reducir parte del coste práctico respecto al enfoque tradicional, manteniendo al mismo tiempo una complejidad asintótica del mismo orden, dominada por la construcción del árbol de productos.

No obstante, los propios autores también señalan una limitación importante. El comportamiento práctico del algoritmo depende de cuántos factores compartidos existan realmente en el conjunto y de cómo estén distribuidos. En particular, pueden aparecer casos más costosos cuando un mismo módulo contiene varios factores que también aparecen en otras claves del conjunto. Aunque este tipo de situaciones no invalida el algoritmo, sí conviene tenerlo en cuenta desde el punto de vista de la implementación.

Desde una perspectiva metodológica, el interés del *Binary Tree Batch GCD* en nuestro trabajo radica en que organiza el problema de forma especialmente adecuada para su implementación eficiente. La estructura binaria facilita la separación del proceso en fases bien definidas: construcción del árbol, acumulación de divisores comunes y enumeración final sobre el conjunto original. Esta organización resulta útil tanto para gestionar la memoria como para plantear posibles estrategias de paralelización, aspectos que serán importantes en la implementación que se describe en el siguiente apartado.

4.4. Implementación desarrollada

Como se detalló en la Sección 4.1, la implementación principal utilizada en este trabajo se ha desarrollado en C++17 empleando la biblioteca GMP para la aritmética multiprecisión. La elección de C++ responde a la necesidad de trabajar con enteros muy grandes de manera eficiente y de disponer de un mayor control sobre el uso de memoria y el rendimiento general de la ejecución. El código está preparado para aprovechar OpenMP para paralelizar la fase de enumeración final cuando el entorno lo permite, aunque conserva la posibilidad de ejecutarse de forma secuencial si el sistema no dispone de esta dependencia instalada.

Siguiendo la lógica del algoritmo descrito en el apartado anterior, la implementación se ha organizado en tres fases claramente diferenciadas. La primera corresponde a la construcción del árbol binario de productos con cálculo de gcd en línea. En esta etapa, los módulos RSA se agrupan por parejas y, en cada nodo, se calcula tanto el producto que se propagará al nivel superior como el gcd entre ambos hijos. Los valores de gcd no triviales se almacenan para ser utilizados posteriormente en la fase de agregación.

Una decisión importante de implementación ha sido liberar los hijos de cada nodo tan pronto como dejan de ser necesarios. De este modo evitamos mantener en memoria más información de la estrictamente necesaria y reducimos el pico de uso de RAM durante la construcción del árbol. Esta decisión es especialmente importante en un problema como este, donde el tamaño de los enteros intermedios crece rápidamente conforme se asciende por los niveles de la estructura.

La segunda fase consiste en la agregación de todos los divisores comunes no triviales en una única variable B . En la versión actual de la implementación, esta agregación se realiza multiplicando secuencialmente los valores almacenados en la fase anterior. Aunque esta parte todavía podría optimizarse más en el futuro, resulta suficiente para validar el comportamiento general del algoritmo y mantener una estructura clara del proceso.

La tercera fase corresponde a la enumeración final sobre el conjunto original de módulos. Para cada N_i , se calcula $\gcd(N_i, B)$ y se comprueba si el resultado es no trivial. Esta fase es especialmente adecuada para una posible paralelización, ya que cada comprobación puede realizarse de forma independiente sobre módulos distintos. Por esta razón, la implementación incluye soporte opcional para OpenMP, aunque el diseño del algoritmo no depende de dicha paralelización.

La Figura 12 resume el flujo de la implementación, desde las fuentes de entrada hasta la salida de factores, tiempos y memoria. Esta separación permite utilizar la misma implementación tanto con conjuntos sintéticos como con módulos extraídos de certificados reales.

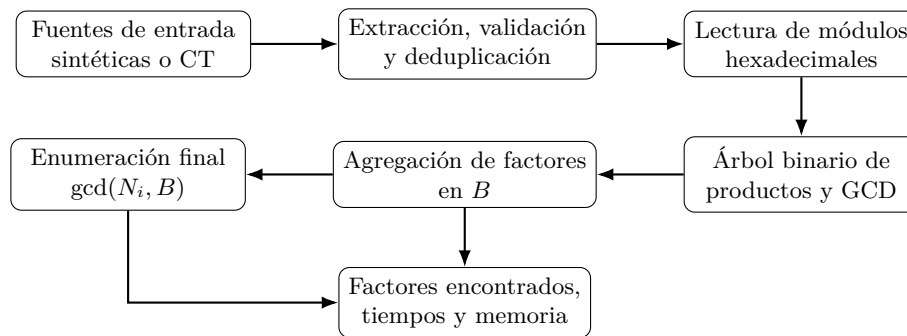


Figura 12: Flujo de la implementación: fuentes sintéticas o CT, preparación de módulos, árbol binario, agregación en B , enumeración final y salida de factores, tiempos y memoria.

Para la fase de pruebas del algoritmo, los datos sintéticos se generaron siguiendo la función `generate_weak_keys` del repositorio original. Para cada configuración se creó un archivo de módulos RSA en formato hexadecimal, uno por línea. Ese mismo archivo se utilizó como entrada para los tres algoritmos comparados: el *Remainder Tree Batch GCD* en Python, el *Binary Tree Batch GCD* en Python y nuestra implementación C++/GMP. Esta decisión garantiza que las diferencias observadas en los resultados se deban al algoritmo y a la implementación, no a variaciones en los datos de entrada.

4.5. Preparación de datos de CT-archive

Además, se ha trabajado con un flujo de preparación de datos reales procedentes de *Certificate Transparency*. El objetivo no es únicamente comprobar que el binario puede leer módulos RSA reales, sino construir un conjunto consolidado a partir de los logs disponibles en *ct-archive*, normalizarlo, eliminar duplicados y ejecutar el algoritmo completo sobre esos datos.

Para ello se ha utilizado el repositorio público *ct-archive*, que recopila volcados de distintos *Certificate Transparency logs* en archivos comprimidos [Geomys, 2026]. El flujo trabaja de forma incremental: descarga un ZIP, inspecciona su contenido, extrae las entradas situadas en `issuer/` y obtiene de cada certificado X.509 su módulo RSA y su emisor.

La elección de `issuer/` responde a una decisión de alcance y viabilidad. Estos directorios proporcionan certificados de autoridades certificadoras e intermediarias directamente procesables, cuyas claves pueden intervenir en numerosas cadenas de certificación. Además, forman un subconjunto más manejable que el contenido completo de los logs y permiten conservar la relación entre cada módulo y su emisor. Los certificados hoja también podrían incorporarse a una auditoría más amplia, pero requerirían pro-

cesar las entradas completas del log.

El campo de módulo se extrae mediante OpenSSL y se almacena en formato hexadecimal. El comando empleado intenta primero interpretar cada certificado como DER y, si falla, repite la operación como PEM:

```
openssl x509 -inform DER -noout -modulus -issuer -in "$cert" 2>/dev/null \
|| openssl x509 -inform PEM -noout -modulus -issuer -in "$cert" 2>/dev/null
```

La opción `x509` selecciona certificados X.509; `-inform` indica la codificación; `-modulus` devuelve el módulo RSA; `-issuer` muestra el emisor; y `-in` identifica el fichero de entrada. La salida conserva pares `modulo||issuer`, lo que permite relacionar un eventual módulo vulnerable con la autoridad certificadora o intermediaria correspondiente.

El proceso incluye una fase de normalización y deduplicación. Muchos certificados pueden contener el mismo módulo RSA por renovaciones, certificados equivalentes o presencia repetida en distintos logs. Ejecutar el algoritmo sobre esas repeticiones no aporta información criptográfica nueva y aumenta artificialmente el tamaño de entrada. Por ello, antes de lanzar el ataque se genera un archivo final con un único módulo hexadecimal por línea.

Durante la extracción se mantiene también un registro del estado de cada ZIP. Los archivos procesados, los que fallan y los que aparentemente no contienen entradas `issuer/` se registran por separado, evitando descartar de forma prematura descargas incompletas o archivos que requieren soporte adecuado para Zip64.

La descarga se realiza mediante HTTP con reanudación, en lugar de basarse directamente en los enlaces Torrent disponibles en la misma fuente. La razón es operativa: el flujo necesita controlar cada ZIP de manera independiente, registrar su estado y liberar espacio cuando deja de ser necesario.

Este paso conecta la parte algorítmica con el objetivo aplicado del trabajo: ejecutar el ataque *all-to-all GCD* sobre certificados reales. La extracción incremental desde *CT logs* genera un conjunto deduplicado de módulos RSA que se analiza con la implementación C++/GMP. La Figura 13 resume el flujo.



Figura 13: Pipeline de datos reales: azul=descarga, verde=extracción, naranja=normalización y rojo=auditoría.

En la memoria no se incluyen notebooks ni listados completos para no romper el hilo expositivo. El capítulo recoge la lógica y las decisiones técnicas principales; sobre esta implementación se apoyan los resultados del capítulo siguiente, con datos sintéticos y módulos RSA reales de *CT logs*.

Capítulo 5

Resultados y discusión

En este capítulo presentamos los resultados obtenidos a partir de la metodología descrita anteriormente. La evaluación se organiza en dos partes. En primer lugar, se estudia el comportamiento de los algoritmos sobre datos sintéticos generados de forma controlada, replicando la estructura experimental de [Pelofske \[2024\]](#). En segundo lugar, se aplica la implementación desarrollada a módulos RSA reales extraídos de *Certificate Transparency logs*. Esta segunda parte incluye la extracción y consolidación de módulos reales, su validación y la ejecución final del ataque *batch GCD* sobre el conjunto deduplicado.

El objetivo principal de la primera parte no es únicamente comprobar que la implementación desarrollada funciona correctamente, sino también comparar su rendimiento con las implementaciones Python del repositorio original. De este modo, la evaluación permite estudiar tres aspectos complementarios: la diferencia entre el enfoque clásico basado en *remainder tree* y el algoritmo *Binary Tree Batch GCD*, el efecto de trasladar este último a C++ con GMP, y la influencia del número de claves débiles introducidas en el conjunto.

5.1. Entorno experimental

Las ejecuciones presentadas en este capítulo se realizaron en un MacBook Pro con arquitectura ARM64, procesador Apple M5 Pro de 18 núcleos y 24 GB de memoria unificada. El sistema utilizado fue macOS sobre el kernel Darwin. La Tabla 1 resume las principales versiones del entorno de software documentado al cierre del trabajo.

Componente	Versión o configuración
Sistema operativo	macOS 26.5, arquitectura ARM64
Python	3.12.11
NumPy	2.3.5
pandas	2.3.3
psutil	7.1.0
PyCryptodome	3.23.0
Compilador	Apple Clang 21.0.0
GMP	6.3.0
OpenMP	LLVM OpenMP 22.1.6

Tabla 1: Entorno de hardware y software utilizado en la evaluación.

Conviene señalar que, aunque se trata de un equipo portátil de prestaciones altas, sus recursos son muy inferiores a los empleados en los trabajos de referencia, que utilizaron hasta 30 núcleos, 180 GB de RAM y en torno a 1 TB de disco [\[Våge, 2022\]](#). Esta diferencia justifica la reducción de escala de la parrilla experimental descrita a continuación.

Para la comparación experimental se utilizaron también las implementaciones Python originales del

repositorio que acompaña a la publicación de Pelofske [2024]. No obstante, dicho repositorio no pudo ejecutarse directamente sin pequeñas adaptaciones de compatibilidad. En particular, fue necesario restaurar el alias `np.product`, eliminado en NumPy 2.0, y resolver las rutas relativas utilizadas por la función de generación de claves débiles, que espera encontrar los primos en un directorio `primes/` relativo a la raíz del repositorio. Estas adaptaciones se implementaron mediante *wrappers* externos que importan el código original sin modificar su lógica algorítmica.

La implementación propia se compiló conforme al estándar C++17, con optimización `-O3`, enlazando las bibliotecas GMP y GMP C++. Cuando se utilizó OpenMP, el número máximo de hilos se fijó en 18, correspondiente a los núcleos disponibles en el equipo. El paralelismo se emplea en la fase de enumeración final de la implementación C++; la construcción del árbol y la agregación de los *gcd* mantienen la organización descrita en el capítulo anterior. Las dos implementaciones de referencia se ejecutaron con el mismo intérprete de Python y sobre los mismos archivos de entrada que la versión C++.

5.2. Configuración de la comparativa sintética

La comparativa sintética se diseñó tomando como referencia a Pelofske [2024, Figura 2]. Dicho experimento analiza dos tamaños de módulo RSA, 1024 y 2048 bits, y tres valores distintos para el número de claves débiles: 2, 100 y 1000. En nuestro caso se mantuvo esta estructura experimental, pero utilizando una parrilla reducida de tamaños de entrada para que la ejecución fuese viable en un entorno local.

En concreto, se utilizaron módulos RSA de 1024 y 2048 bits, generados como producto de primos de 512 y 1024 bits respectivamente. Para cada tamaño de módulo se probaron los valores $WEAK = 2$, $WEAK = 100$ y $WEAK = 1000$, y para cada una de estas configuraciones se evaluaron conjuntos de $N = 2000$, $N = 5000$ y $N = 10000$ módulos. Estos valores de N son menores que los empleados en el artículo original, donde se llega hasta 75000 módulos para claves RSA-2048 y hasta 120000 módulos para claves RSA-1024. Esta reducción responde al coste computacional de ejecutar las implementaciones Python originales en un portátil, especialmente en el caso del método basado en *remainder tree*.

Los tres algoritmos comparados fueron los siguientes:

- *Remainder Tree Batch GCD*, utilizando la implementación Python original del repositorio de Pelofske.
- *Binary Tree Batch GCD*, utilizando también la implementación Python original del mismo repositorio.
- *Binary Tree Batch GCD* en C++17 con GMP, correspondiente a la implementación desarrollada en este trabajo.

Para garantizar que la comparación fuese homogénea, cada combinación de tamaño de módulo, valor de $WEAK$ y valor de N generó un único archivo de módulos en hexadecimal. Los tres algoritmos se ejecutaron después sobre ese mismo archivo de entrada. De este modo, las diferencias observadas en tiempo de ejecución y consumo de memoria no se deben a variaciones en los datos, sino a la implementación y al algoritmo utilizado.

Cada algoritmo se ejecutó una vez por configuración. El tiempo se midió como tiempo de pared (*wall time*), es decir, desde el lanzamiento hasta la finalización del proceso. El consumo de memoria se estimó mediante el máximo de memoria residente (*peak RSS*) observado para el proceso y sus hijos. Por tanto, los valores presentados son mediciones puntuales de las 18 configuraciones completadas, no estimaciones estadísticas obtenidas a partir de repeticiones. Las aceleraciones medias se calcularon como la media aritmética de los cocientes de tiempo de cada configuración.

5.3. Validación funcional

Antes de analizar los tiempos de ejecución, se comprobó que los tres algoritmos identificaban los mismos módulos vulnerables en cada configuración experimental. Esta validación es importante porque la implementación C++ no solo debe ser más rápida, sino también producir resultados compatibles con las implementaciones Python de referencia.

Los resultados fueron consistentes en todas las combinaciones evaluadas: para cada conjunto sintético, los tres algoritmos detectaron exactamente el mismo número de módulos con factores compartidos. Por ejemplo, para $WEAK = 2$ se detectaron 4 módulos vulnerables; para $WEAK = 100$, los valores estuvieron alrededor de 200; y para $WEAK = 1000$, oscilaron entre 1747 y 1970 según el tamaño del conjunto. Además, para cada factor p recuperado se comprobó que dividía al módulo correspondiente ($N \bmod p = 0$) y que el cofactor N/p era un entero mayor que uno.

El número de módulos vulnerables detectados no tiene por qué coincidir exactamente con $WEAK$, ya que la función de generación introduce módulos que comparten un factor con otros ya existentes. Por tanto, aparecen tanto los módulos añadidos como los originales con los que comparten factor. Lo relevante para la validación es que las tres implementaciones coincidieron exactamente en todos los casos.

5.4. Comparación de tiempos de ejecución

La Figura 14 muestra los tiempos de ejecución de los tres algoritmos. Como en el experimento de Pelofske, las filas distinguen el tamaño del módulo RSA y las columnas representan los valores de $WEAK$. La escala vertical es logarítmica para apreciar mejor las diferencias entre la implementación C++ y las versiones Python.

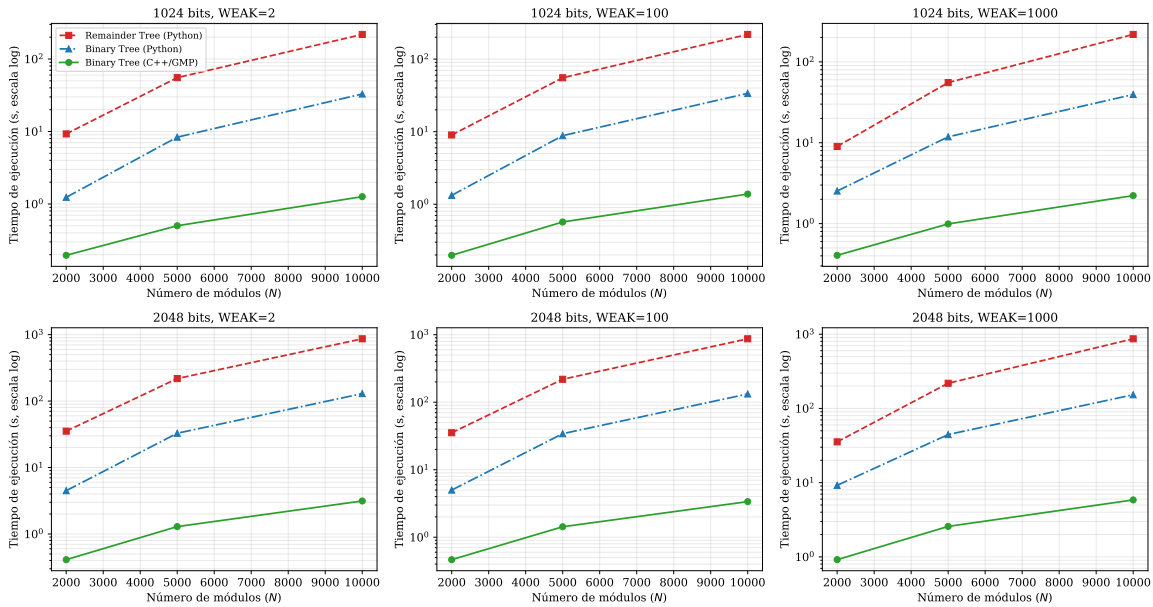


Figura 14: Comparación del tiempo de ejecución de los tres algoritmos evaluados sobre datos sintéticos: *Remainder Tree Batch GCD* en Python, *Binary Tree Batch GCD* en Python y *Binary Tree Batch GCD* en C++/GMP.

Los resultados muestran una tendencia clara. En primer lugar, la implementación Python del *Binary Tree Batch GCD* mejora de forma consistente al método clásico basado en *Remainder Tree Batch GCD*. En promedio, el algoritmo de árbol binario fue aproximadamente 6,1 veces más rápido que el

método clásico, con aceleraciones entre 3,6 y 7,9 según la configuración.

En segundo lugar, la implementación C++ con GMP reduce de forma muy significativa el tiempo de ejecución respecto a las dos versiones Python. Frente al *Remainder Tree Batch GCD*, la aceleración media fue de aproximadamente 116,5 veces, alcanzando un máximo de 275,3 veces. Frente al *Binary Tree Batch GCD* en Python, la implementación C++ fue en promedio 18,7 veces más rápida, con un máximo de 41,3 veces.

Por tanto, los resultados deben interpretarse en dos niveles: la comparación entre las dos implementaciones Python refleja principalmente la diferencia entre algoritmos, mientras que la comparación con C++/GMP mide la mejora práctica obtenida al trasladar el algoritmo seleccionado a una implementación más eficiente.

La Tabla 2 recoge algunos casos representativos de la comparativa. El caso más exigente dentro de nuestra parrilla experimental corresponde a módulos RSA de 2048 bits, $N = 10000$ y $WEAK = 1000$. En esa configuración, el método clásico tardó 866,819 segundos, la versión Python del algoritmo de Pelofske tardó 152,803 segundos y la implementación C++/GMP completó la ejecución en 5,851 segundos.

Bits	WEAK	N	Remainder Python	Binary Python	Binary C++/GMP
1024	100	10000	218,579 s	33,721 s	1,381 s
2048	100	10000	870,017 s	132,484 s	3,368 s
2048	1000	10000	866,819 s	152,803 s	5,851 s

Tabla 2: Tiempos representativos de la comparativa sintética.

La forma de estas curvas es comparable con la Figura 2 de Pelofske: el coste aumenta con N , el método basado en *remainder tree* permanece por encima de *Binary Tree Batch GCD* y la separación se mantiene para los dos tamaños de clave y los tres valores de $WEAK$ [Pelofske, 2024]. La comparación no es punto por punto, porque nuestra figura añade la implementación C++/GMP y se detiene en $N = 10000$, mientras que Pelofske alcanza 75000 módulos para RSA-2048 y 120000 para RSA-1024. Por tanto, se ha replicado la estructura y la tendencia del experimento, no su escala máxima.

El crecimiento observado dentro de la parrilla evaluada también permite comparar cómo responde cada implementación al aumentar la entrada. Para módulos RSA de 2048 bits y $WEAK = 1000$, pasar de $N = 2000$ a $N = 10000$, es decir, multiplicar por cinco el número de módulos, elevó el tiempo de la implementación C++/GMP de 0,917 a 5,851 segundos, un factor de 6,38. En la versión Python de *Binary Tree Batch GCD*, el tiempo pasó de 9,201 a 152,803 segundos, un factor de 16,61, mientras que el método clásico pasó de 35,440 a 866,819 segundos, un factor de 24,46. Estos cocientes describen las mediciones de esta configuración concreta y no deben interpretarse como una estimación empírica de la complejidad asintótica, pero muestran que la separación entre las implementaciones aumenta en los tamaños considerados.

Los resultados son coherentes con la motivación de Pelofske. El algoritmo *Binary Tree Batch GCD* reduce el coste práctico frente al método clásico al evitar la construcción explícita de un *remainder tree*. Nuestra implementación confirma además que, al trasladar esta lógica a C++ y utilizar una biblioteca optimizada para aritmética multiprecisión como GMP, la mejora práctica es mucho mayor.

5.5. Comparación de consumo de memoria

Además del tiempo de ejecución, también se midió el pico de memoria RAM consumida por cada algoritmo. La Figura 15 muestra los resultados obtenidos.

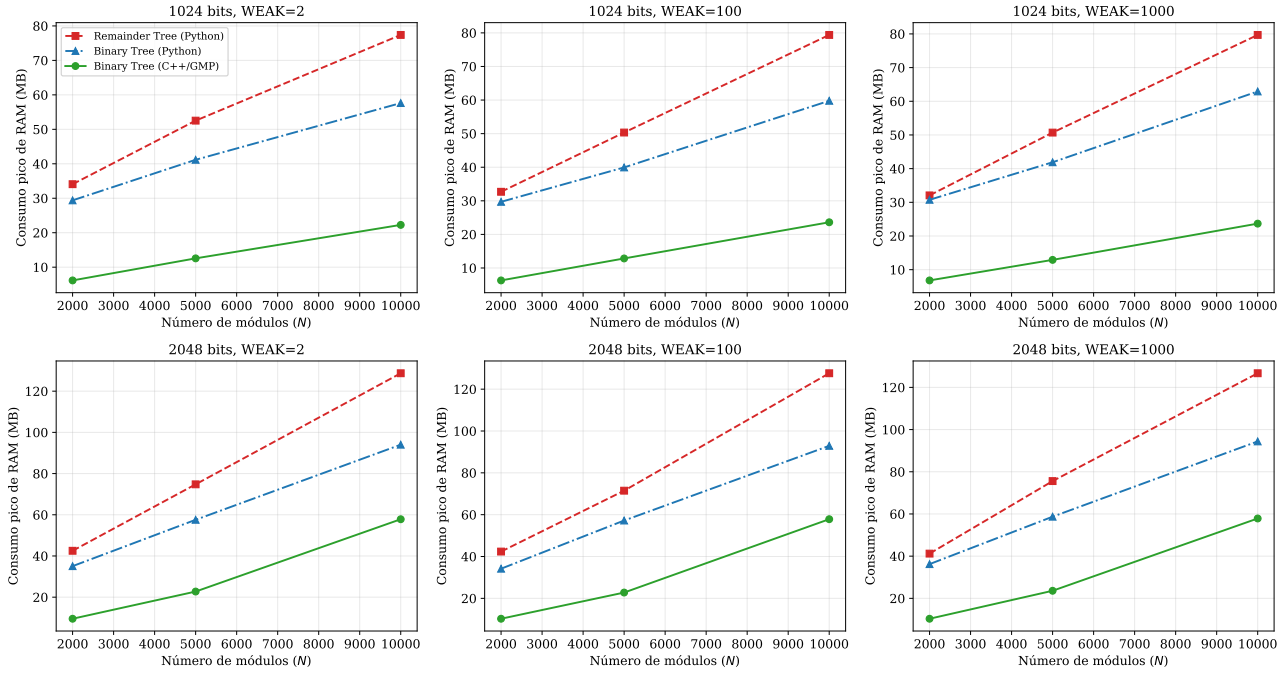


Figura 15: Comparación del consumo pico de memoria de los tres algoritmos evaluados: en verde Binary Tree (C++/GMP), en azul Binary Tree (Python) y en rojo la implementación clásica en Python.

La tendencia observada es también favorable a la implementación C++/GMP. El mayor pico de memoria registrado para el método clásico en Python fue de 128,70 MB, mientras que la versión Python del algoritmo de Pelofske alcanzó 94,39 MB. En cambio, la implementación C++/GMP tuvo un máximo de 57,86 MB en el conjunto de experimentos. Tomando estos máximos como referencia descriptiva, la versión C++ redujo el pico en un 55,0 % respecto al método clásico y en un 38,7 % respecto a *Binary Tree Batch GCD* en Python.

Esta diferencia no debe interpretarse únicamente como una consecuencia del lenguaje utilizado. También está relacionada con la forma en que la implementación gestiona los enteros intermedios durante la construcción del árbol. En particular, en la versión C++ se liberan los hijos de cada nodo cuando dejan de ser necesarios, reduciendo así la cantidad de datos mantenidos simultáneamente en memoria. Aunque el tamaño de los enteros crece inevitablemente al ascender por el árbol, esta gestión permite contener el pico de RAM.

5.6. Efecto del número de claves débiles

El parámetro WEAK permite observar cómo afecta la cantidad de factores compartidos al comportamiento de los algoritmos. Los resultados muestran que el método clásico basado en *remainder tree* apenas cambia al variar este parámetro. Por ejemplo, con módulos RSA de 2048 bits y $N = 10000$, los tiempos fueron 864,321 segundos para $WEAK = 2$, 870,017 segundos para $WEAK = 100$ y 866,819 segundos para $WEAK = 1000$.

En cambio, el efecto de WEAK sí es más visible en el algoritmo de árbol binario. En la misma configuración de 2048 bits y $N = 10000$, la versión Python pasó de 129,501 segundos con $WEAK = 2$ a 152,803 segundos con $WEAK = 1000$. La implementación C++/GMP pasó de 3,139 segundos a 5,851 segundos.

En términos relativos, estos cambios representan un incremento del 18,0 % para la versión Python y del 86,4 % para la implementación C++/GMP. El porcentaje mayor de la versión C++ debe in-

interpretarse junto con su menor tiempo absoluto: incluso en la configuración con más claves débiles, finalizó en 5,851 segundos, muy por debajo de los 152,803 segundos de la versión Python y de los 866,819 segundos del método clásico. Como se trata de una única medición por configuración, estos porcentajes describen los puntos observados y no una distribución estadística del rendimiento.

Este comportamiento coincide con lo descrito en la literatura: el algoritmo *Binary Tree Batch GCD* resulta especialmente ventajoso cuando el número de factores compartidos es pequeño en comparación con el tamaño total del conjunto. A medida que aumenta el número de módulos débiles, la ventaja se reduce parcialmente, aunque el algoritmo sigue siendo claramente más eficiente que el enfoque clásico en todas las configuraciones evaluadas.

5.7. Resultados sobre datos reales

5.7.1. Extracción y consolidación de módulos reales

Una vez validado el comportamiento sobre datos sintéticos y comparado con las implementaciones de referencia, el siguiente paso consiste en aplicar el algoritmo a módulos RSA reales. Esta parte es la que conecta directamente con el objetivo aplicado del trabajo: utilizar el nuevo algoritmo para buscar factores RSA compartidos en certificados extraídos de *CT logs*.

La fuente principal utilizada para esta fase fue *ct-archive*, una colección pública de volcados de *Certificate Transparency logs* organizada por log y por archivos ZIP [Geomys, 2026]. El procedimiento no descarga toda la colección de forma permanente. En su lugar, procesa los archivos comprimidos de forma incremental: descarga un ZIP, extrae únicamente los certificados situados bajo la carpeta **issuer/**, obtiene con OpenSSL el módulo RSA y el emisor de cada certificado, registra los resultados intermedios y libera el archivo comprimido cuando deja de ser necesario.

Esta decisión reduce de forma considerable el volumen de datos que debe conservarse en disco y delimita el análisis a un subconjunto relevante y operativamente manejable. Los certificados disponibles en **issuer/** corresponden a autoridades certificadoras o intermediarias y pueden procesarse directamente para recuperar sus claves públicas y conservar el contexto del emisor. El conjunto obtenido no representa, por tanto, todos los módulos RSA publicados en *Certificate Transparency*: los certificados hoja contenidos en las entradas completas de los logs también podrían aportar módulos adicionales. Su incorporación requeriría descargar y decodificar esas entradas mediante un procedimiento distinto. Los logs marcados en el repositorio como carentes de **issuer/** no se procesan por la vía empleada en este trabajo.

El proceso de extracción se diseñó de forma conservadora. Cada ZIP queda clasificado como procesado, fallido o sin carpeta **issuer/**. Los casos aparentemente vacíos se revisan antes de descartarse, porque una descarga parcial o un problema al inspeccionar archivos grandes puede hacer que un ZIP válido parezca no contener la ruta esperada. Durante la ejecución se observaron incidencias de este tipo, especialmente en archivos grandes que requerían soporte adecuado para Zip64. Por ello se revisaron de forma específica logs como `ct_sectigo_elephant2025h2` y `ct_sectigo_tiger2025h2`, evitando clasificar como inútiles archivos que en realidad estaban incompletos o no habían sido inspeccionados correctamente.

Los módulos extraídos de los distintos logs de *ct-archive* se consolidaron en un único fichero. Todos ellos se normalizaron al mismo formato hexadecimal y se deduplicaron globalmente. Esta deduplicación es necesaria porque un mismo módulo puede aparecer en distintos certificados o en distintos logs. Repetirlo varias veces no aporta información criptográfica nueva para el ataque *batch GCD* y solo aumentaría artificialmente el tamaño del problema.

El conjunto consolidado final contiene 54.278 módulos únicos brutos. Tras aplicar una validación básica, que descarta valores no plausibles como módulos RSA por tamaño o paridad, quedan 54.275 módulos RSA plausibles. En concreto, se descartaron tres valores: dos por ser pares y uno por tener una longitud inferior al umbral considerado. La Tabla 3 resume el conjunto real utilizado.

Elemento	Valor
Módulos únicos brutos	54278
Módulos RSA plausibles	54275
Módulos descartados en validación	3
Logs de <i>ct-archive</i> resumidos	20
Logs incompletos al finalizar la extracción	0

Tabla 3: Resumen del conjunto real de módulos RSA consolidado.

La distribución por tamaño de los módulos plausibles se muestra en la Tabla 4. La mayoría de los módulos corresponden a claves RSA de 2048 bits, aunque también aparecen claves de 1024, 3072 y 4096 bits, además de algunos tamaños ligeramente distintos.

Tamaño del módulo	Módulos RSA plausibles
1024 bits	9
2045 bits	1
2048 bits	53539
3072 bits	204
4036 bits	1
4096 bits	521

Tabla 4: Distribución por tamaño de los módulos RSA plausibles analizados.

Si se intentase analizar este conjunto mediante una estrategia ingenua de todos los pares, habría que calcular el máximo común divisor para todos los pares posibles de módulos:

$$\frac{54275 \cdot 54274}{2} = 1472860675$$

comparaciones. Aunque este tamaño es mucho menor que los conjuntos masivos considerados en estudios previos, sigue siendo suficiente para justificar el uso de un enfoque *batch GCD* frente a la comparación exhaustiva par a par.

5.7.2. Ejecución del ataque sobre módulos reales

El conjunto de 54.275 módulos RSA plausibles se procesó con la implementación C++/GMP desarrollada en este trabajo. En esta ejecución se utilizó soporte OpenMP, con 18 hilos disponibles en el entorno local. El programa cargó los módulos normalizados, construyó el árbol binario de productos con cálculo de *gcd* en línea, agregó los divisores comunes encontrados en la variable *B* y realizó la enumeración final sobre todos los módulos.

La Tabla 5 recoge los principales resultados de la ejecución. Durante la construcción del árbol no se obtuvo ningún *gcd* no trivial, por lo que la variable agregada *B* quedó con longitud de 1 bit. En consecuencia, la fase final tampoco detectó claves vulnerables. El tiempo total del algoritmo fue de 23.756,22 milisegundos, es decir, aproximadamente 23,8 segundos.

El resultado debe interpretarse de forma precisa. En el conjunto real analizado no se han encontrado factores RSA compartidos ni claves vulnerables. Esto no implica que no existan factores compartidos en otros *CT logs* ni en el conjunto completo de certificados publicados en Internet, sino únicamente que no se han detectado dentro del subconjunto extraído, normalizado y deduplicado en este trabajo.

Desde el punto de vista metodológico, la ejecución confirma que la implementación puede utilizarse sobre datos reales y no solo sobre conjuntos sintéticos. La extracción de módulos, la validación

Elemento de la ejecución	Valor
Módulos RSA analizados	54275
Hilos OpenMP utilizados	18
Niveles del árbol construido	16
<i>GCDs</i> no triviales en fase 1	0
Claves vulnerables detectadas	0
Tiempo fase 1	23753,25 ms
Tiempo fase 2	0,00 ms
Tiempo fase 3	2,96 ms
Tiempo total del algoritmo	23756,22 ms

Tabla 5: Resultado del ataque *Binary Tree Batch GCD* sobre módulos reales.

de formato, la eliminación de duplicados y el ataque *batch GCD* se integran en un flujo completo y documentado sobre certificados procedentes de *Certificate Transparency*.

5.8. Discusión

Los resultados obtenidos permiten extraer varias conclusiones. En primer lugar, la reproducción reducida del experimento confirma la ventaja práctica del algoritmo *Binary Tree Batch GCD* frente al enfoque clásico basado en *remainder tree*. Aunque nuestra parrilla de tamaños es más pequeña que la del artículo original, la tendencia observada es consistente: el algoritmo de árbol binario reduce de forma clara el tiempo de ejecución.

En segundo lugar, la implementación C++/GMP desarrollada en este trabajo mejora de manera notable a las implementaciones Python originales. Esta mejora era esperable por el uso de una biblioteca multiprecisión optimizada y por el mayor control sobre la gestión de memoria, pero los resultados cuantifican su impacto práctico. En los casos más grandes evaluados, la diferencia entre completar el análisis en varios minutos o en pocos segundos es relevante de cara a su aplicación sobre conjuntos reales.

En tercer lugar, la variación de **WEAK** muestra que el rendimiento del algoritmo de árbol binario depende en cierta medida del número de factores compartidos presentes en el conjunto. Esto es importante para el problema real, ya que en colecciones de certificados auténticas se espera que los factores compartidos sean eventos poco frecuentes. En ese escenario, precisamente, el algoritmo de Pelofske debería comportarse de forma especialmente favorable.

La principal limitación de esta fase experimental es el tamaño de la parrilla de N y el alcance del conjunto real analizado. La reproducción completa de la figura del artículo requeriría alcanzar 75000 módulos para RSA-2048 y 120000 módulos para RSA-1024, además de repetir cada punto para los tres valores de **WEAK** y, en nuestro caso, para tres implementaciones distintas. Ejecutar esa campaña completa en un portátil no resulta práctico, especialmente por el coste de las implementaciones Python. Del mismo modo, los 54.275 módulos reales analizados permiten validar y aplicar el flujo completo, pero no agotan el espacio de certificados existente en todos los *CT logs*. Por ello, los resultados reales deben entenderse como una auditoría acotada al conjunto extraído y consolidado en este trabajo.

Capítulo 6

Conclusiones y limitaciones

En este Trabajo de Fin de Máster se ha estudiado la detección de factores compartidos en claves públicas RSA extraídas de *Certificate Transparency logs*. El punto de partida ha sido una línea de trabajo previa basada en el análisis masivo de certificados, iniciada por la tesis de Våge [Våge, 2022], y una propuesta algorítmica más reciente, el *Binary Tree Batch GCD* [Pelofske, 2024].

La primera conclusión del trabajo es que el algoritmo *Binary Tree Batch GCD* constituye una alternativa práctica al enfoque clásico basado en *product tree* y *remainder tree*. La reproducción reducida del experimento de Pelofske confirma que, incluso en un entorno local y con tamaños de entrada menores que los del artículo original, la tendencia es clara: el algoritmo de árbol binario reduce de forma consistente el tiempo de ejecución frente al método clásico.

La segunda conclusión es que la implementación tiene un impacto decisivo. La versión C++17/GMP desarrollada mejora de forma muy notable a las implementaciones Python utilizadas como referencia. Esta mejora no se debe únicamente al cambio de lenguaje, sino también a la gestión explícita de los enteros intermedios y a la liberación de memoria durante la construcción del árbol. Para el objetivo aplicado del trabajo, esta diferencia es importante, porque convierte el algoritmo en una herramienta más adecuada para analizar conjuntos reales de certificados.

La tercera conclusión es que el comportamiento observado al variar el número de claves débiles encaja con la motivación del algoritmo. El *Binary Tree Batch GCD* resulta especialmente interesante cuando los factores compartidos son poco frecuentes en comparación con el tamaño total del conjunto. Este escenario es precisamente el esperable en una colección real de certificados, ya que la mayoría de claves deberían ser seguras, y los factores compartidos (si aparecen) deberían ser casos excepcionales.

Además, se ha aplicado el flujo completo sobre datos reales. A partir de los certificados de emisores extraídos de los distintos logs disponibles en *ct-archive* se consolidó un conjunto de 54.278 módulos únicos brutos. Tras la validación de tamaño y paridad, quedaron 54.275 módulos RSA plausibles, que fueron procesados por la implementación C++/GMP. En este conjunto no se detectaron factores compartidos no triviales ni claves vulnerables.

La obtención de este conjunto real se basó principalmente en *ct-archive*. El flujo desarrollado descarga los archivos comprimidos de los logs seleccionados, extrae los certificados presentes en la carpeta `issuer/`, obtiene de cada uno el módulo RSA y el emisor, y genera tanto una relación `modulo|issuer` como un conjunto deduplicado de módulos para alimentar al binario C++.

Esta fase ha puesto de manifiesto que la obtención de módulos reales es una parte relevante del problema. No basta con disponer del algoritmo: es necesario controlar descargas parciales, revisar archivos marcados erróneamente como carentes de `issuer/`, tratar correctamente archivos ZIP grandes, deduplicar los módulos y conservar información contextual suficiente para interpretar los resultados.

Este proceso convierte la implementación en una herramienta más cercana a una auditoría real sobre *CT logs*.

La ejecución final sobre los 54.275 módulos RSA plausibles se completó en aproximadamente 23,8 segundos en el entorno local utilizado, empleando OpenMP. Este resultado muestra que, para conjuntos de este tamaño, la implementación desarrollada permite realizar la auditoría de forma práctica sin necesidad de comparar todos los pares de módulos. No obstante, el resultado debe interpretarse de manera acotada: no se han encontrado factores compartidos en el conjunto analizado, pero esto no demuestra la ausencia de vulnerabilidades en todos los certificados publicados en *CT logs*.

Los resultados están sujetos a varias limitaciones. En primer lugar, la comparativa sintética reproduce la estructura del experimento de Pelofske, pero utiliza tamaños máximos de $N = 10000$, inferiores a los del trabajo original. Esta reducción permitió ejecutar las tres implementaciones sobre las mismas entradas en un equipo local, aunque limita la comparación directa con los puntos de mayor escala del artículo. Además, cada algoritmo se ejecutó una sola vez por configuración, por lo que los tiempos y picos de memoria deben interpretarse como mediciones puntuales y no como estimaciones acompañadas de variabilidad estadística. Aun con esta limitación, la ordenación de las tres implementaciones fue la misma en las 18 configuraciones completadas: el algoritmo de árbol binario fue más rápido que el método clásico y la implementación C++/GMP obtuvo el menor tiempo.

En segundo lugar, el conjunto real está condicionado por la fuente y el procedimiento de extracción. El flujo se centra en los certificados disponibles en *issuer/*, por lo que no incluye los logs que carecen de esa estructura ni las entradas completas de todos los certificados publicados en CT. Si se eliminase esta restricción, la auditoría cubriría una fracción mayor del ecosistema y el resultado negativo tendría un alcance empírico más amplio. Con los datos disponibles, la afirmación válida sigue siendo necesariamente más concreta: no se detectaron factores RSA compartidos entre los 54.275 módulos plausibles y únicos analizados.

Por último, la agregación de los *gcd* no triviales en la variable B se realiza secuencialmente. Esta decisión no afectó de manera apreciable a la ejecución real, porque la primera fase no produjo divisores comunes no triviales, y tampoco impidió completar correctamente los experimentos sintéticos. Su coste podría adquirir mayor importancia en un conjunto con una concentración elevada de claves vulnerables; resolver esa limitación mediante una agregación equilibrada o paralela reduciría ese posible cuello de botella, pero no altera los resultados medidos en este trabajo.

En conjunto, el TFM aporta una implementación funcional y validada de *Binary Tree Batch GCD*, una comparación documentada con las implementaciones Python de referencia y un flujo completo para extraer, normalizar y auditar módulos RSA procedentes de *Certificate Transparency logs*. Los resultados confirman la ventaja práctica del algoritmo frente al enfoque clásico dentro de las configuraciones evaluadas y muestran que la implementación C++/GMP permite aplicar esta técnica de forma eficiente a un conjunto real de certificados.

Como trabajos futuros y problemas abiertos, quedan varias líneas de mejora. Por un lado, sería interesante ampliar el conjunto de certificados analizados, incorporando no solo certificados de *issuer/*, sino también certificados hoja extraídos de las entradas completas de los logs. Por otro lado, se podría profundizar en la optimización computacional de la implementación, especialmente en la paralelización de fases que todavía se ejecutan de forma secuencial y en la evaluación sobre conjuntos de tamaño más cercano a los utilizados en la literatura. De este modo, el trabajo desarrollado aquí puede entenderse como una base para auditorías más amplias sobre claves RSA publicadas en *CT logs*.

Referencias

- J.-P. Aumasson. *Serious cryptography: a practical introduction to modern encryption*. No Starch Press, Inc, 2024.
- E. Barker. Recommendation for key management: Part 1 – general. NIST Special Publication 800-57 Part 1 Revision 5, National Institute of Standards and Technology, 2020.
- Geomys. ct-archive, 2026.
<https://github.com/geomys/ct-archive> Repositorio de volcados de Certificate Transparency logs. Consultado el 7 de junio de 2026.
- T. Granlund y the GMP Development Team. *GNU MP: The GNU Multiple Precision Arithmetic Library*, 6.3.0 edition, 2023.
<https://gmplib.org/manual/>
- M. Hastings, J. Fried y N. Heninger. Weak keys remain widespread in network devices. In *Proceedings of the 2016 Internet Measurement Conference (IMC)*, pages 49–63, 2016.
- N. Heninger, Z. Durumeric, E. Wustrow, y J. A. Halderman. Mining your ps and qs: Detection of widespread weak keys in network devices. In *Proceedings of the 21st USENIX Security Symposium (USENIX Security 12)*, pages 205–220, 2012.
- J. D. Kilgallin y R. Vasko. Factoring RSA keys in the IoT era. First IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA), 2019.
- B. Laurie, A. Langley, E. Kasper, E. Messeri, y R. Stradling. Certificate transparency version 2.0. Technical Report RFC 9162, Internet Engineering Task Force, 2021.
- A. J. Menezes, P. C. van Oorschot y S. A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996.
- National Institute of Standards and Technology. Recommendation for the entropy sources used for random bit generation. Technical Report NIST Special Publication 800-90B, U.S. Department of Commerce, Washington, D.C., 2001a.
- National Institute of Standards and Technology. Transition to post-quantum cryptography standards. NIST Internal Report IR 8547 (Initial Public Draft), NIST, 2024.
- National Institute of Standards and Technology. Advanced Encryption Standard (AES). FIPS 197-upd1, National Institute of Standards and Technology, November 2001b.
<https://doi.org/10.6028/NIST.FIPS.197-upd1> Updated May 9, 2023.
- OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*, 2021.
<https://www.openmp.org/spec-html/5.2/openmp.html>
- E. Pelofske. An efficient all-to-all GCD algorithm for low entropy RSA key factorization. Cryptology ePrint Archive, Paper 2024/699, 2024.
<https://eprint.iacr.org/2024/699>

- E. Rescorla. The transport layer security (tls) protocol version 1.3. Technical Report RFC 8446, Internet Engineering Task Force, 2018.
- R. L. Rivest, A. Shamir y L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- RSA Laboratories. The RSA Factoring Challenge, 1991.
https://en.wikipedia.org/wiki/RSA_Factoring_Challenge Challenge proposed to encourage research on integer factorization.
- P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- I. T. Union. Recommendation itu-t x.500: Open systems interconnection. Standard, ITU, 1988.
<https://www.itu.int/rec/T-REC-X.500-198811-S>
- H. F. Våge. Finding shared rsa factors in the certificate transparency logs. Master’s thesis, Department of Informatics, University of Bergen, May 2022.